

I. Frequency-domain inverse wave propagation problem

Anushi R. Sadam

Dr. Omar Ghattas

CSE393P: Computational & Variational Methods for Inverse Problems

30 April 2025

Assignment 04

I) Frequency-domain inverse wave propagation problem

$$\min_m \phi(m) := \frac{1}{2} \sum_{i=1}^{N_f} \sum_{j=1}^{N_s} \int_{\Gamma} (u_{ij}(m) - d_{ij})^2 ds + \frac{\beta}{2} \int_{\Omega} \nabla m \cdot \nabla m dx$$

forward problem:

$$-\Delta u_{ij} - k_{0,i}^2 (1-m) u_{ij} = k_{0,i}^2 m u_{ij}^{inc} \text{ in } \Omega \quad i=1 \dots N_f, j=1 \dots N_s$$

$$\frac{\partial u_{ij}}{\partial n} = 0 \quad \text{on } \Gamma$$

$$k_{0,i} = \frac{\omega_i}{c_0}, \quad d_{ij}(x) := \text{data measurements}$$

I (i) find the infinite-dimensional gradient of ϕ w.r.t m .
assume $N_f = N_s = 1$.

First, we find the weak formulation of the forward problem:

let $u = u_{ij}$, since $N_f = N_s = 1$.

$$\text{then, } -\Delta u - k_{0,i}^2 (1-m) u = k_{0,i}^2 m u^{inc} \text{ in } \Omega \quad (1a)$$

$$\frac{\partial u}{\partial n} = 0 \quad \text{on } \Gamma \quad (1b)$$

Multiply (1a) by a test function $p \in H^1$ & integrate over domain:

$$\int_{\Omega} p (\Delta u - k_{0,i}^2 (1-m) u) dx = \int_{\Omega} (k_{0,i}^2 m u^{inc}) p dx \quad \text{on } \Omega \quad \forall p \in H^1$$

$$\int_{\Gamma} \frac{\partial u}{\partial n} p = 0 \quad \text{on } \Gamma$$

Integrating by parts:

$$\begin{aligned} \int_{\Omega} (-\nabla \cdot (p \nabla u) + \nabla p \nabla u - K_{0,i}^2 (1-m) u p) dx &= \int_{\Omega} K_{0,i}^2 m u^{inc} p dx \\ \int_{\Omega} \nabla \cdot (p \nabla u) dx &= \int_{\Gamma} (p \nabla u) \cdot \vec{n} ds \quad (\text{divergence theorem}) \\ \int_{\Omega} \nabla p \nabla u - K_{0,i}^2 (1-m) u p dx - \int_{\Gamma} (p \nabla u) \cdot \vec{n} ds &= \int_{\Omega} K_{0,i}^2 m u^{inc} p dx \\ \nabla u \cdot \vec{n} &= 0 \end{aligned}$$

The weak form of the forward problem is:

$$\int_{\Omega} [\nabla p \nabla u - K_{0,i}^2 [(1-m) u + m u^{inc}] p] dx = 0$$

To form the Lagrangian:

$$\begin{aligned} \mathcal{L}(u, m, p) &= \frac{1}{2} \int_{\Gamma} (u - d)^2 ds + \frac{\beta}{2} \int_{\Omega} \nabla m \cdot \nabla m dx \\ &\quad + \int_{\Omega} \nabla p \nabla u - K_{0,i}^2 [(1-m) u + m u^{inc}] p dx \end{aligned} \quad \Bigg\} \mathcal{L}(m)$$

i) To get the forward problem

$$\delta_p \mathcal{L} = 0 \quad \forall \hat{p} \in H^1$$

weak form of fwd problem

$$\delta_p \mathcal{L} = \int_{\Omega} \nabla \hat{p} \nabla u - K_{0,i}^2 [(1-m) u + m u^{inc}] \hat{p} dx = 0 \quad \forall \hat{p} \in H^1$$

given m , find u s.t. $\downarrow$ is satisfied

Integrate by parts:

3

$$\delta_p \mathcal{L} = \int_{\Omega} \hat{p} [-\Delta u - K_{0,i}^2 (1-m)u + m u^{inc}] dx + \int_{\Gamma} \hat{p} (\nabla u \cdot \vec{n}) ds = 0$$

since $\hat{p}$ is arbitrary:

strong form

$$\left. \begin{array}{l} -\Delta u - K_{0,i}^2 (1-m)u = K_{0,i}^2 m u^{inc} \quad \text{on } \Omega \\ \nabla u \cdot \vec{n} = 0 \quad \text{on } \Gamma \end{array} \right\} \begin{array}{l} \text{of fwd} \\ \text{problem} \\ \text{natural BC} \end{array}$$

ii) For adjoint problem:

given u, m , find p s.t. $\downarrow$ is satisfied

$$\delta_u \mathcal{L} = 0 \quad \forall \hat{u} \in H^1$$

weak form of adjoint problem

$$\delta_u \mathcal{L} = \int_{\Gamma} (u-d) \hat{u} ds + \int_{\Omega} \nabla p \cdot \nabla \hat{u} - K_{0,i}^2 (1-m) \hat{u} p dx = 0 \quad \forall \hat{u} \in H^1$$

Integrate by parts:

$$\int_{\Gamma} (u-d) \hat{u} ds + \int_{\Omega} \hat{u} [\Delta p - K_{0,i}^2 (1-m)p] dx + \int_{\Gamma} \hat{u} (\nabla p \cdot \vec{n}) ds = 0 \quad \forall \hat{u} \in H^1$$

$$\Rightarrow \left. \begin{array}{l} \Delta p - K_{0,i}^2 (1-m)p = 0 \quad \text{on } \Omega \\ \nabla p \cdot \vec{n} = (u-d) \quad \text{on } \Gamma \end{array} \right\} \begin{array}{l} \text{strong form of} \\ \text{adjoint problem} \end{array}$$

note $p \in H^1$, not H^0 , since data observations (1d) are made on the boundary. The BCs for p "pop-out" naturally.

ii) For the gradient:

 $\delta_m \mathcal{L} :=$ weak form of gradient

$$\delta_m \mathcal{L} = \beta \int_{\Omega} \nabla m \cdot \nabla \hat{m} dx + \int_{\Omega} (K_{0,i}^2 u \hat{m} + K_{0,i}^2 u^{inc} \hat{m}) p dx \quad \forall \hat{m} \in H^1$$

given u, m, p evaluate gradient s.t.:

$$= \left[\beta \int_{\Omega} \nabla \hat{m} \cdot \nabla m dx + \int_{\Omega} \hat{m} (K_{0,i}^2 u + K_{0,i}^2 u^{inc}) p dx \right] \quad \forall \hat{m} \in H^1$$

Integrate by parts for strong form:

weak form of gradient

$$\begin{array}{l} \text{in weak} \\ \text{form} \end{array} \quad \begin{array}{l} \text{gradient} \\ \langle g(m), \hat{m} \rangle = \int_{\Omega} \hat{m} (\beta \nabla m + K_{0,i}^2 (u + u^{inc}) p) dx \\ \quad + \int_{\Gamma} \beta \hat{m} (\nabla m \cdot \vec{n}) dx \end{array} \quad \forall \hat{m} \in H^1$$

In strong form, the gradient is:

$$\begin{cases} -\beta \Delta m + \beta K_{0,i}^2 (u - u^{inc}) p & \text{on } \Omega \\ \beta \nabla m \cdot \vec{n} & \text{on } \Gamma \end{cases}$$

Lagrange multiplier

enforcing fwd/adjoint eqns

I (2) find the inf-dim hessian action in direction $\tilde{m}$
Construct Lagrangian for hessian, $\mathcal{L}^H(u, m, p; \tilde{u}, \tilde{m}, \tilde{p})$

$$\begin{aligned} \mathcal{L}^H(u, m, p; \tilde{u}, \tilde{m}, \tilde{p}) = & \beta \int_{\Omega} \nabla \tilde{m} \cdot \nabla m \, dx + \int_{\Omega} \tilde{m} (K_{0,i}^2 u + K_{0,i}^2 u^{inc}) p \, dx \\ & + \int_{\Gamma} (u - d) \tilde{u} \, ds + \int_{\Omega} \nabla p \cdot \nabla \tilde{u} - K_{0,i}^2 (1 - m) \tilde{u} p \, dx \\ & + \int_{\Omega} \nabla \tilde{p} \cdot \nabla u - K_{0,i}^2 (1 - m) u + m u^{inc} \tilde{p} \, dx \end{aligned}$$

iv) for the incremental forward problem

$$\delta_p \mathcal{L}^H = 0 \quad \forall \hat{p} \in H^1$$

$$\int_{\Omega} \tilde{m} K_{0,i}^2 (u - u^{inc}) \hat{p} + \nabla \hat{p} \cdot \nabla \tilde{u} - K_{0,i}^2 (1 - m) \tilde{u} \hat{p} \, dx = 0 \quad \forall \hat{p} \in H^1$$

$$\begin{aligned} \int_{\Omega} \hat{p} [\tilde{m} K_{0,i}^2 (u - u^{inc}) - \Delta \tilde{u} - K_{0,i}^2 (1 - m) \tilde{u}] \, dx = 0 \quad \forall \hat{p} \in H^1 \\ + \int_{\Gamma} \hat{p} \nabla \tilde{u} \cdot \vec{n} \, ds \end{aligned}$$

$\hat{p}$ is arbitrary $\Rightarrow$

$$\begin{aligned} \text{Strong form of } & \Delta \tilde{u} + K_{0,i}^2 \tilde{u} (1 - m) = \tilde{m} K_{0,i}^2 (u - u^{inc}) \quad \text{on } \Omega \\ \text{Incremental } & \nabla \tilde{u} \cdot \vec{n} = 0 \quad \text{on } \Gamma \end{aligned}$$

fwd Same BCs as for u

v) For the incremental adjoint problem:

5

$$\delta_u \mathcal{I}^H = 0 \quad \forall \hat{u} \in H^1$$

$$\int_{\Omega} \tilde{m} k_{0,i}^2 \hat{u} + \beta \tilde{p} \nabla \hat{u} - k_{0,i}^2 [(1-m) \hat{u}] \tilde{p} \, dx + \int_{\Gamma} \tilde{u} \tilde{u} \, ds = 0$$

$$\int_{\Omega} \hat{u} [\tilde{m} k_{0,i}^2 \tilde{p} - \Delta \tilde{p} - k_{0,i}^2 (1-m) \tilde{p}] \, dx + \int_{\Gamma} \hat{u} (\tilde{u} + \nabla \tilde{p} \cdot \vec{n}) \, ds = 0 \quad \forall \hat{u} \in H^1$$

$$\begin{aligned} \Delta \tilde{p} + k_{0,i}^2 (1-m) \tilde{p} &= \tilde{m} k_{0,i}^2 \quad \text{on } \Omega \\ \tilde{p} \cdot \vec{n} &= -\tilde{u} \quad \text{on } \Gamma \end{aligned} \quad \left. \begin{array}{l} \text{Strong form of incremental} \\ \text{adjoint} \end{array} \right\}$$

vi) For the hessian action $(\hat{m}, H(m) \tilde{m})$

$$\delta_m \mathcal{I}^H = \int_{\Omega} \beta \nabla \tilde{m} \cdot \nabla \tilde{m} \, dx + \int_{\Omega} \left[k_{0,i}^2 \hat{m} \tilde{u} \tilde{p} + k_{0,i}^2 \hat{m} u \tilde{p} - k_{0,i}^2 \hat{m} u^{inc} \tilde{p} \right] \, dx$$

find $H(m) \tilde{m}$ s.t. $\delta_m \mathcal{I}^H = 0$ is true $\forall \hat{m} \in H^1$

weak form of Hessian action: $(\hat{m}, H(m) \tilde{m})$:

$$\delta_m \mathcal{I}^H = \int_{\Omega} \beta \nabla \hat{m} \cdot \nabla \tilde{m} + k_{0,i}^2 (\hat{m} \tilde{u} \tilde{p} + \hat{m} u \tilde{p} - \hat{m} u^{inc} \tilde{p}) \, dx \quad \forall \hat{m} \in H^1$$

Integrate by parts for strong form:

$$\delta_m \mathcal{I}^H = \int_{\Omega} \hat{m} [-\beta \Delta \tilde{m} + (\tilde{u} \tilde{p} + u \tilde{p} - u^{inc} \tilde{p}) k_{0,i}^2] \, dx + \int_{\Gamma} \beta \hat{m} \nabla \tilde{m} \cdot \vec{n} \, ds$$

since $\hat{m}$ is arbitrary:

$$\begin{aligned} & \left[\begin{array}{ll} -\beta \Delta \tilde{m} + (\tilde{u} \tilde{p} + u \tilde{p} - u^{inc} \tilde{p}) k_{0,i}^2 & \text{on } \Omega \\ \beta \nabla \tilde{m} \cdot \vec{n} & \text{on } \Gamma \end{array} \right] \quad \left. \begin{array}{l} \text{Strong form of} \\ \text{Hessian action} \end{array} \right\} \end{aligned}$$

1(3) find the inf-dim gradient for an arbitrary number of sources & frequencies ($N_s \neq N_f \neq 1$)

$$\mathcal{L}(m, u_{ij}, p_{ij}) = \frac{1}{2} \sum_i \sum_j \int_{\Gamma} (u_{ij}(m) - d_{ij})^2 \, ds + \frac{\beta}{2} \int_{\Omega} \nabla m \cdot \nabla m \, dx$$

for full number of sources/frequencies $+ \sum_i \sum_j \int_{\Omega} \nabla p_{ij} \cdot \nabla u_{ij} - k_{0,i}^2 [(1-m) u_{ij} + m u^{inc}] p_{ij} \, dx$

- The forward problem:

$\delta_{p_{ij}} \mathcal{L}_f = 0 \quad \forall \hat{p}_{ij} \in H'$ for each i, j source, frequency.

$$\delta_{p_{ij}} \mathcal{L}_f = \left[\int_{\mathcal{R}} \nabla p_{ij} \cdot \nabla u_{ij} - k_{0,i}^2 [(1-m) u_{ij} + m u^{inc}] p_{ij} dx = 0 \quad \forall \hat{p}_{ij} \in H' \right]$$

In strong form:

$$\left\{ \begin{array}{l} -\Delta u_{ij} - k_{0,i}^2 [(1-m) u_{ij}] = k_{0,i}^2 m u^{inc} \quad \text{in } \mathcal{R} \\ \nabla u_{ij} \cdot \vec{n} = 0 \quad \text{on } \Gamma \end{array} \right.$$

So, for the fwd problem, there are $N_f \times N_s$ PDE constraints of form $\delta_{p_{ij}} \mathcal{L}_f = 0 \quad \forall \hat{p}_{ij} \in H'$.

- The adjoint problem

$\delta u_{ij} \mathcal{L}_f = 0 \quad \forall \hat{u}_{ij} \in H'$ for each i, j source, frequency

$$\delta u_{ij} \mathcal{L}_f = \left[\int_{\Gamma} (u_{ij} - d_{ij}) \hat{u}_{ij} ds + \int_{\mathcal{R}} \nabla p_{ij} \cdot \nabla \hat{u}_{ij} - k_{0,i}^2 (1-m) \hat{u}_{ij} p_{ij} dx = 0 \quad \forall \hat{u}_{ij} \in H' \right]$$

In strong form:

$$\left\{ \begin{array}{l} -\Delta p_{ij} - k_{0,i}^2 (1-m) p_{ij} = 0 \quad \text{on } \mathcal{R} \\ \nabla p_{ij} \cdot \vec{n} = -(u_{ij} - d_{ij}) \quad \text{on } \Gamma \end{array} \right.$$

For the adjoint problem, there are $N_f \times N_s$ PDE constraints of form $\delta u_{ij} \mathcal{L}_f = 0 \quad \forall \hat{u}_{ij} \in H'$

- The gradient

$\delta m \mathcal{L}_f, \quad \forall \hat{m} \in H'$

$$\delta m \mathcal{L}_f = \beta \int_{\mathcal{R}} \nabla \hat{m} \cdot \nabla m dx + \sum_i \sum_j^{N_f N_s} \int_{\mathcal{R}} k_{0,i}^2 (u_{ij} - u^{inc}) \hat{m} p_{ij} dx$$

in weak form, $(G_f(m), \hat{m}) = \nabla$

In strong form:

$$\begin{aligned} -f \Delta m + \sum_i^{N_f} \sum_j^{N_s} K_{o,ij}^2 (u_{ij} - u_{inc}) p_{ij} & \text{ on } \Omega \\ \beta \nabla m \cdot n & \text{ on } \Gamma \end{aligned}$$

To compute a single gradient $\uparrow$, $N_f \times N_s$ state equations & $N_f \times N_s$ adjoint equations must be solved for $N_f \times N_s$ u_{ij} values & $N_f \times N_s$ p_{ij} values ($N_f \times N_s$ combinations of ij 'th index).

II. The steepest descent method for solving the Helmholtz inverse problem

Derivations:

II) Steepest Descent for solving Helmholtz problem

$$1. \quad -\Delta u - k_0^2(1 - \tanh(m))u = k_0^2 \tanh(m) u^{inc} \quad \text{in } \Omega$$

$$\frac{\partial u}{\partial n} = 0 \quad \text{on } \Gamma$$

Convert to weak form of the fwd problem: $p \in H^1$, adjoint

$$\int_{\Omega} \nabla u \cdot \nabla p - k_0^2(1 - \tanh(m))up \, dx = \int_{\Omega} k_0^2 \tanh(m) u^{inc} p \, dx$$

$$\int_{\Omega} [\nabla u \cdot \nabla p - k_0^2(1 - \tanh(m))up] \, dx - \int_{\Gamma} p \left[\frac{\partial u}{\partial n} \right] \, ds$$

$$= \int_{\Omega} k_0^2 \tanh(m) u^{inc} p \, dx$$

The weak form of the forward problem is, then:
find the state variable u , given parameter m , such that:

$$\forall p \in H^1, \quad \int_{\Omega} \nabla u \cdot \nabla p - k_0^2(1 - \tanh(m))up \, dx - \int_{\Omega} k_0^2 \tanh(m) u^{inc} p \, dx = 0$$

The Lagrangian, $\mathcal{L}(u, m, p) =$

$$\mathcal{L}(u, m, p) = \frac{1}{2} \int_{\Gamma} (u - d)^2 \, ds + \frac{\beta}{2} \int_{\Omega} \nabla m \cdot \nabla m \, dx$$

$$+ \int_{\Omega} \nabla p \cdot \nabla u - p u k_0^2(1 - \tanh(m)) \, dx - \int_{\Omega} p u^{inc} k_0^2 \tanh(m) \, dx$$

• The forward problem:

$$\delta_p \mathcal{L} = 0 \quad \forall \hat{p} \in H^1:$$

find u s.t.:

$$\delta_p \mathcal{L} = \int_{\Omega} \nabla \hat{p} \cdot \nabla u - \hat{p} u k_0^2(1 - \tanh(m)) \, dx - \int_{\Omega} \hat{p} u^{inc} k_0^2 \tanh(m) \, dx$$

$$= 0 \quad \forall \hat{p} \in H^1$$

- The adjoint problem:
Find p s.t. $\delta_u \mathcal{L} = 0 \quad \forall \hat{u} \in H^1$

$$\delta_u \mathcal{L} = \int_{\Gamma} (u-d) \hat{u} \, ds + \int_{\Omega} \nabla p \cdot \nabla \hat{u} - p \hat{u} k_0^2 (1 - \tanh(m)) \, dx = 0$$

$$\forall \hat{u} \in H^1$$

- The gradient

$$\begin{aligned} \delta_m \left(\int_{\Omega} \tanh(m) \, dx \right) &= \left. \frac{d}{d\varepsilon} \int_{\Omega} \tanh(m + \varepsilon \hat{m}) \, dx \right|_{\varepsilon=0} \\ &= \int_{\Omega} \hat{m} \cdot (1 - \tanh^2(m + \varepsilon \hat{m})) \Big|_{\varepsilon=0} \, dx \\ &= \int_{\Omega} \hat{m} (1 - \tanh^2(m)) \, dx \end{aligned}$$

$$\delta_m \mathcal{L} = \beta \int_{\Omega} \nabla \hat{m} \cdot \nabla m \, dx + \int_{\Omega} \hat{m} [p u k_0^2 (1 - \tanh^2(m)) - p u^{inc} k_0^2 (1 - \tanh^2(m))] \, dx$$

$$\begin{aligned} \delta_m \mathcal{L} &= \beta \int_{\Omega} \nabla \hat{m} \cdot \nabla m \, dx + \int_{\Omega} \hat{m} p k_0^2 (1 - \tanh^2(m)) [u - u^{inc}] \, dx \\ &= \langle G_1(m), \hat{m} \rangle \end{aligned}$$

For Steepest Descent:

find $\hat{m}_k$ s.t.:

$$\int_{\Omega} \hat{m}_k \hat{m} \, dx = - \langle G_1(m), \hat{m} \rangle \quad \forall \hat{m} \in H^1$$

update direction $\nearrow$ requires u_k, p_k based on $m = m_k$

To get the boundary conditions on p , integrate the adjoint equation by parts

$$\int_{\Gamma} (u-d) \hat{u} \, ds + \int_{\Omega} \hat{u} (-\Delta p - p k_0^2 (1 - \tanh(m))) \, dx + \int_{\Gamma} \hat{u} \nabla p \cdot \vec{n} \, ds = 0 \quad \forall \hat{u} \in H^1$$

Since $\hat{u}$ is arbitrary, in strong form, the adjoint eqn:

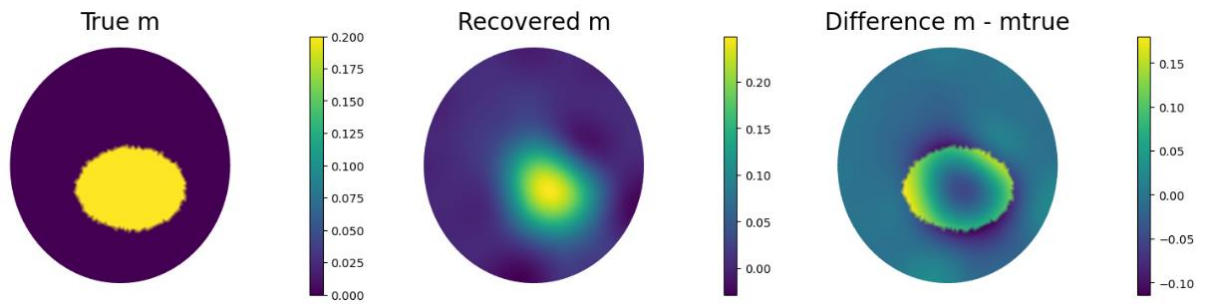
$$\begin{aligned} \Delta p + p k_0^2 (1 - \tanh(m)) &= 0 & \text{on } \Omega \\ \nabla p \cdot \vec{n} &= -(u-d) & \text{on } \Gamma \end{aligned}$$

II-2. If using a total variation regularizer $R_{TV}^\delta = \beta \int_{\Omega} (\nabla m \cdot \nabla m + \delta)^{1/2} \, dx$,

only the variation w.r.t m , the weak form of the gradient, will change: (let $\delta = 10^{-6}$)

$$\begin{aligned} \mathcal{L}(u, m, p) &= \frac{1}{2} \int_{\Gamma} (u-d)^2 \, ds + \beta \int_{\Omega} (\nabla m \cdot \nabla m + \delta)^{1/2} \, dx \\ &\quad + \int_{\Omega} [p \nabla u - p u k_0^2 (1 - \tanh(m))] \, dx \\ &\quad - \int_{\Omega} p u^{\text{inc}} k_0^2 \tanh(m) \, dx \\ \delta_m \mathcal{L} &= \beta \int_{\Omega} (\nabla m \cdot \nabla m + \delta)^{-1/2} \cdot \nabla m \cdot \nabla \hat{m} \, dx \quad \leftarrow \text{1st variation of } m \\ &\quad + \int_{\Omega} \hat{m} p k_0^2 (1 - \tanh^2(m)) [u - u^{\text{inc}}] \, dx \\ &= \langle G_1(m), \hat{m} \rangle \quad \forall \hat{m} \in H^1 \end{aligned}$$

1. Report the solution of the inverse problem and the number of required iterations (for a 1000 relative reduction in the norm of the gradient) for the following cases.
 - a. Yes, the problem converges with steepest descent (Tikhonov regularization, $k_0 = 5$) in under 1,000 iterations. It takes 719 iterations for convergence to the below solution:



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.00472$.

Cost function:

```
# Define cost function
def cost(u, d, m, beta):
    reg = 0.5*beta * dl.assemble( ufl.inner(dl.grad(m), ufl.grad(m))*ufl.dx )
    misfit = 0.5 * dl.assemble( (u-d)**2*ufl.dx )
    return [reg + misfit, misfit, reg]
```

Gradient:

```
def pde_varf(u,m,p):
    return ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx \
        - k0squared*(dl.Constant(1.) - ufl.tanh(m) ) * u * p * ufl.dx \
        - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx
```

```
bc_state = []
```

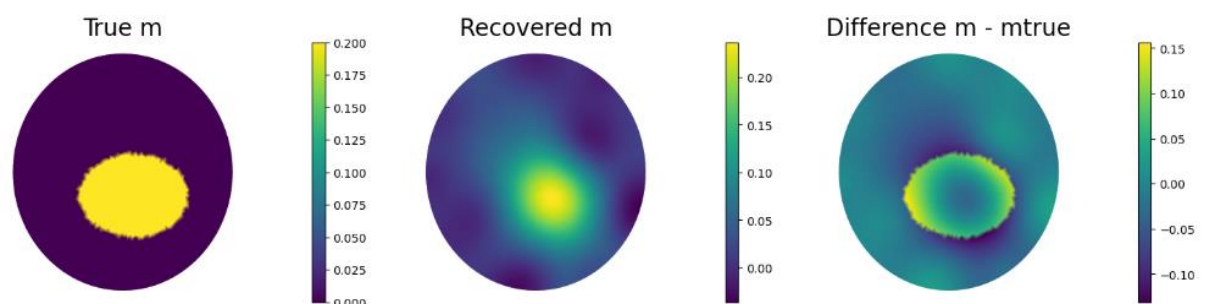
```
# solve the adjoint problem
adj_A, adjoint_RHS = dl.assemble_system(a_adj, l_adj)
dl.solve(adj_A, p.vector(), adjoint_RHS)

# evaluate the gradient
MG = dl.assemble(grad_misfit + grad_reg)
dl.solve(P, g, MG)

# calculate the norm of the gradient
grad_norm2 = g.inner(MG)
gradnorm = np.sqrt(grad_norm2)
```

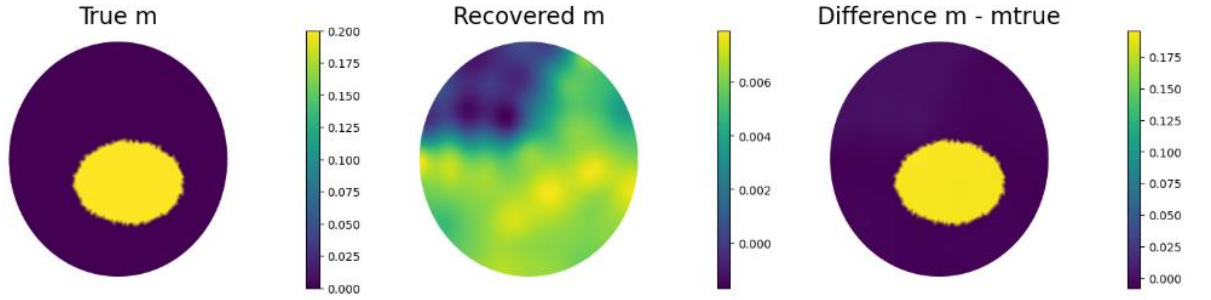
For further details on code, please refer to the Appendix.

- b. With a H^1 -inner product preconditioning, steepest descent (Tikhonov regularization, $k_0 = 5$) converges to the below solution in 402 iterations, which is much faster than the L^2 -inner product preconditioning.



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.00565$, which is slightly higher (likely negligible) than the L^2 -inner product preconditioned solution.

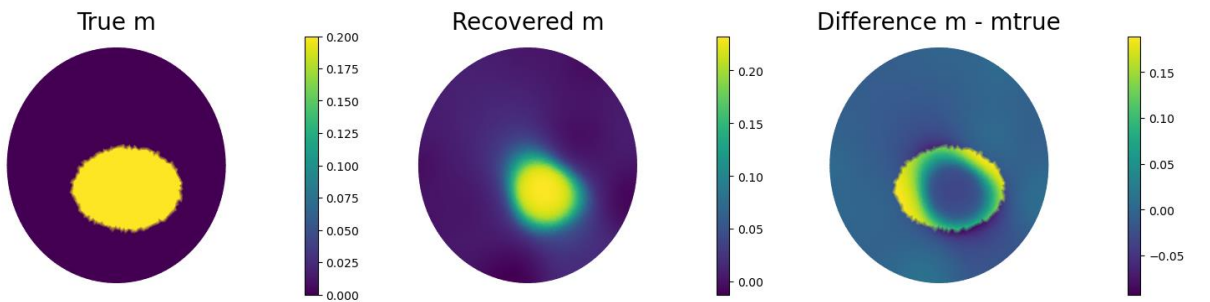
- c. With a higher wave number $k_0 = 10$ (H^1 -inner product preconditioning, Tikhonov regularization) steepest descent does not converge within 1000 iterations, and does not recover the parameter field (m) well. There is little-to-no recovery with 1000 iterations, so we assume that further iterations will not improve convergence.



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.0202$, which is about 1 order of magnitude more error than for a lower wavenumber of $k_0 = 5$.

2. Steepest Descent for Helmholtz with Total Variation

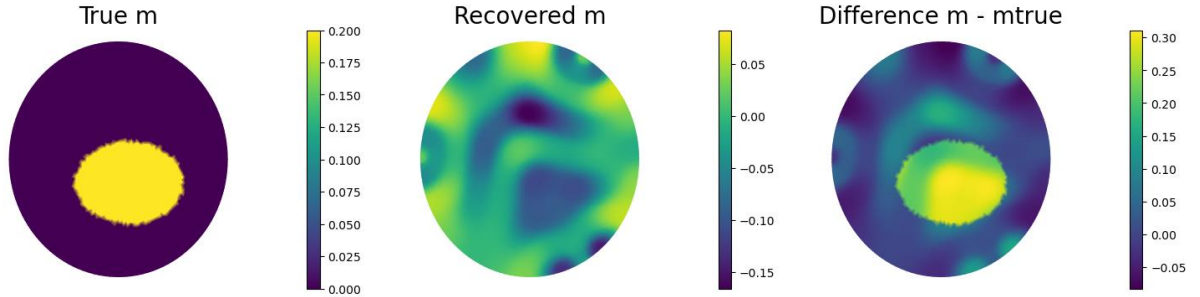
The problem does not converge even after 2000 iterations when Total Variation regularization is implemented (steepest descent, $k_0 = 5$, smoothing parameter .01 for TV regularization). The final result is:



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.00576$, which is about the same as the Tikhonov regularized solutions for $k_0 = 5$. However, notice that with TV regularization, a sharper parameter field is able to be reconstructed, compared to the blurred recovered parameter field with Tikhonov regularization. Further refinement could be achieved by varying the regularization parameter (currently set at 10^{-5}) and the smoothing parameter for the regularization (set at .01). TV leads to a highly irregular/ill-posed loss function, so convergence constraints might need to be relaxed. Note that the gradient norm of TV regularized loss changes from about 10^{-2} to 10^{-5} , indicating

substantial optimization and well recovery of the parameter field. (The gradient reduction tolerance is 10^{-3} , which is almost achieved).

For $k_0 = 10$, just as with Tikhonov regularization, the parameter field is not recovered well:



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.0435$, which is about 1 order of magnitude more error than for a lower wavenumber of $k_0 = 5$, and the solution does not converge in 2000 iterations.

Cost function:

```
delta = .01
# Define cost function
def cost(u, d, m, beta):
    reg = beta * dl.assemble( ufl.sqrt(ufl.inner(ufl.grad(m), ufl.grad(m)) + delta)*ufl.dx )
    misfit = 0.5 * dl.assemble( (u-d)**2*ufl.dx )
    return [reg + misfit, misfit, reg]
```

Gradient:

```
def pde_varf(u, m, p, u_inc):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(dl.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# weak form for setting up the state equation
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx

# weak form for setting up the adjoint equations
a_adj = (ufl.inner( ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx

# weak form for gradient
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(ufl.tanh(m))) * (u - u_inc)) * ufl.dx
grad_reg = (dl.Constant(beta) * ufl.inner(ufl.grad(m_test), ufl.grad(m)) / ufl.sqrt(ufl.inner(ufl.grad(m), ufl.grad(m)) + dl.Constant(delta))) * ufl.dx

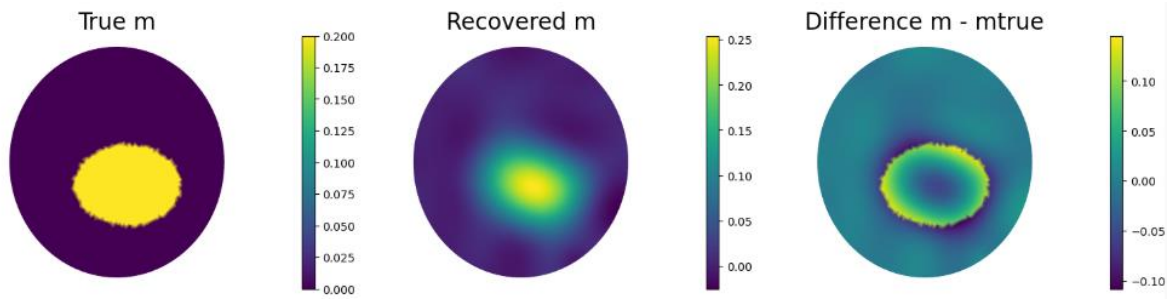
# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# matrix in parameter space
Kvarf = ufl.inner(m_trial, m_test) * ufl.dx + ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
K = dl.assemble(Kvarf)

m.assign(m0)

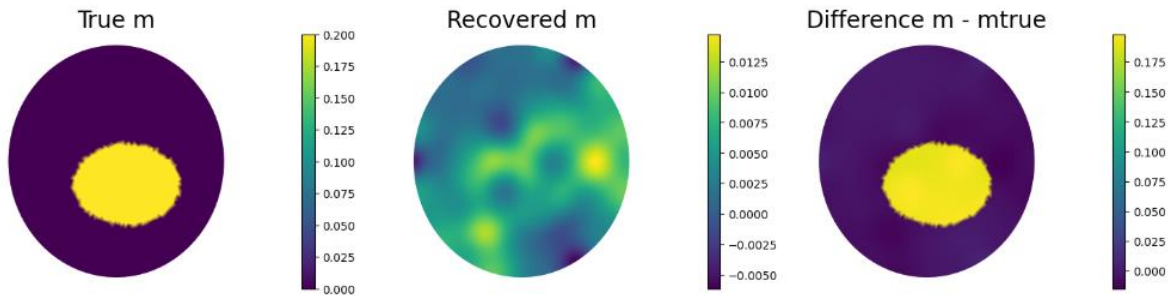
# solve state equation
A, state_b = dl.assemble_system (a_state, L_state)
dl.solve(A, u.vector(), state_b)
```

3. With multiple sources superposed independently, $k_0 = 5$, Tikhonov regularization, and steepest descent, we achieve the below solution:



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.00353$, which is marginally better than the error when the sources are superimposed (II.1.a, error of 0.00472). The results seem to have more resolution/crispness (more similar to the Total Variation regularization in II.2 than Tikhonov), than when the sources are superimposed.

If a $k_0 = 10$ is used, steepest descent does not converge in 1000 iterations, and the parameter field recovery is still quite poor:



The L^2 error $\|m - m_{\text{true}}\|_2^2 \approx 0.0203$, which is slightly better than the error when the superimposed (II.1.c, error of 0.0435), but still extremely bad in terms of parameter recovery accuracy. Overall, the results seem slightly better than when the sources are superimposed, but they generally take some more computational expense.

III. The inexact Newton-CG method for solving the Helmholtz inverse problem

Derivations:

III) Inexact Newton CG for Helmholtz

$$\begin{aligned} \mathcal{L}^H(u, m, p; \tilde{u}, \tilde{m}, \tilde{p}) = & \beta \int_{\Omega} \nabla \tilde{m} \cdot \nabla m \, dx + \int_{\Omega} \tilde{m} p k_0^2 (u - u^{\text{inc}}) (1 - \tanh^2(m)) \, dx \\ & + \int_{\Gamma} (u - d) \tilde{u} \, ds + \int_{\Omega} \nabla p \nabla \tilde{u} - p \tilde{u} k_0^2 (1 - \tanh^2(m)) \, dx \\ & + \int_{\Omega} \nabla \tilde{p} \nabla u - \tilde{p} u k_0^2 (1 - \tanh^2(m)) \, dx \\ & - \int_{\Omega} \tilde{p} u^{\text{inc}} k_0^2 \tanh(m) \, dx \end{aligned}$$

$$\delta_p \mathcal{L}^H = 0 \quad \forall \hat{p} \in H^1$$

incremental forward problem (weak form)

$$\begin{cases} \int_{\Omega} \tilde{m} \hat{p} k_0^2 (u - u^{\text{inc}}) (1 - \tanh^2(m)) \, dx \\ + \int_{\Omega} \nabla \hat{p} \nabla \tilde{u} - \hat{p} \tilde{u} k_0^2 (1 - \tanh^2(m)) \, dx = 0 \quad \forall \hat{p} \in H^1 \end{cases}$$

write in form of: $A_k \hat{u}_k = -C_k \hat{m}_k$ per previous notation, $\hat{u}_k = \tilde{u}$
 $\hat{m}_k = \tilde{m}$

$$A_k \hat{u} = \int_{\Omega} \nabla \hat{p} \nabla \tilde{u} - p \tilde{u} k_0^2 (1 - \tanh^2(m)) \, dx$$

$$C_k \hat{m} = \int_{\Omega} \tilde{m} \hat{p} k_0^2 (u - u^{\text{inc}}) (1 - \tanh^2(m)) \, dx$$

trial test

$$\delta_u \mathcal{L}^H = 0 \quad \forall \hat{u} \in H^1$$

incremental adjoint problem (weak form)

$$\begin{cases} \int_{\Omega} \tilde{m} p k_0^2 \hat{u} (1 - \tanh^2(m)) \, dx + \int_{\Gamma} \hat{u} \tilde{u} \, ds \\ + \int_{\Omega} \nabla \tilde{p} \nabla \hat{u} - \tilde{p} \hat{u} k_0^2 (1 - \tanh^2(m)) \, dx = 0 \quad \forall \hat{u} \in H^1 \end{cases}$$

write in form $A_k^T \hat{p}_k = -(W_{uu} \hat{u}_k + W_{um} \hat{m}_k)$ 12

$$\hat{p}_k = \tilde{p} \quad \hat{u}_k = \tilde{u} \quad \hat{m}_k = \tilde{m}$$

$$A_k^T \tilde{p} = \int_{\Omega} -\tilde{p} \tilde{u} k_0^2 (1 - \tanh^2(m)) + \nabla \tilde{p} \cdot \nabla \tilde{u} \, dx$$

$$W_{uu} \tilde{u} = \int_{\Gamma} \tilde{u} \tilde{u} \, ds$$

$$W_{um} \tilde{m} = \int_{\Omega} \tilde{m} p k_0^2 \tilde{u} (1 - \tanh^2(m)) \, dx$$

$$\delta_m \mathcal{L}^H = (\tilde{m}, H(m) \tilde{m})$$

$$= \beta \int_{\Omega} \nabla \tilde{m} \cdot \nabla \tilde{m} \, dx + \int_{\Omega} -p \tilde{u} k_0^2 (1 - \tanh^2(m)) \tilde{m} \, dx$$

$$+ \int_{\Omega} \tilde{p} u k_0^2 (1 - \tanh^2(m)) \tilde{m} \, dx$$

$$- \int_{\Omega} \tilde{p} u^{inc} k_0^2 (1 - \tanh^2(m)) \tilde{m} \, dx$$

$$- \int_{\Omega} \tilde{m} p k_0^2 (u - u^{inc}) \cdot 2 \tanh(m) (1 - \tanh^2(m)) \tilde{m} \, dx$$

$$H_k \tilde{m} = R \tilde{m} + (W_{mm} + C_k^T A_k^{-T} (W_{uu} A_k^{-1} (k - W_{um}) - W_{mu} A_k^{-1} k)) \cdot \tilde{m}$$

includes test function product $(\tilde{m})$

$$R \tilde{m} = \beta \int_{\Omega} \nabla \tilde{m} \cdot \nabla \tilde{m} \, dx$$

$$W_{mm} \tilde{m} = - \int_{\Omega} \tilde{m} p k_0^2 (u - u^{inc}) \cdot 2 \tanh(m) (1 - \tanh^2(m)) \tilde{m} \, dx$$

For a total variation regularization: can add a δ 13

$$\begin{aligned} \mathcal{L}(u, m, p) = & \frac{1}{2} \int_{\Gamma} (u-d)^2 ds + \beta \int_{\Omega} (\nabla m \cdot \nabla m)^{1/2} dx \\ & + \int_{\Omega} [p \nabla u - p u k_0^2 (1 - \tanh(m))] dx \\ & - \int_{\Omega} p u^{inc} k_0^2 \tanh(m) dx \end{aligned}$$

$$\mathcal{L}^H(u, m, p; \tilde{u}, \tilde{m}, \tilde{p}) = \beta \int_{\Omega} (\nabla m \cdot \nabla m + \delta)^{-1/2} \cdot \nabla m \cdot \nabla \tilde{m} dx + \dots$$

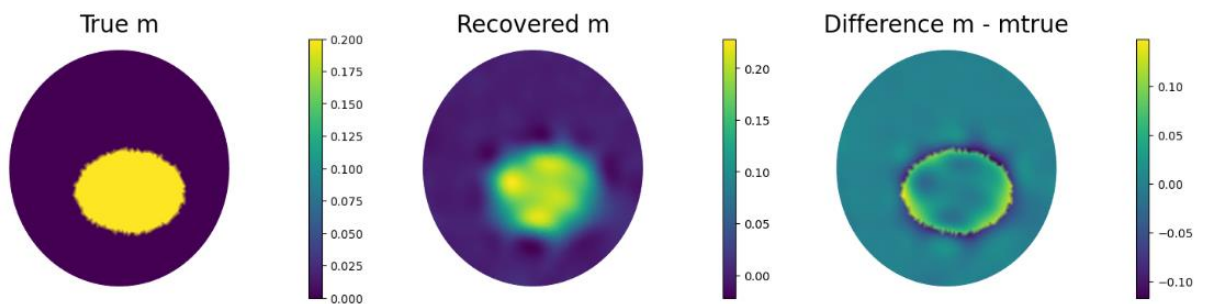
$$\delta_m \mathcal{L}^H = \beta \int_{\Omega} \nabla \tilde{m} \left[\Delta m (\nabla m \cdot \nabla m + \delta)^{-1/2} + \nabla m \cdot \frac{-1}{2} (\nabla m \cdot \nabla m + \delta)^{-3/2} \right. \\ \left. \cdot 2 \cdot \nabla m \cdot \nabla \tilde{m} \right] \dots$$

$\mathcal{R} \tilde{m}$

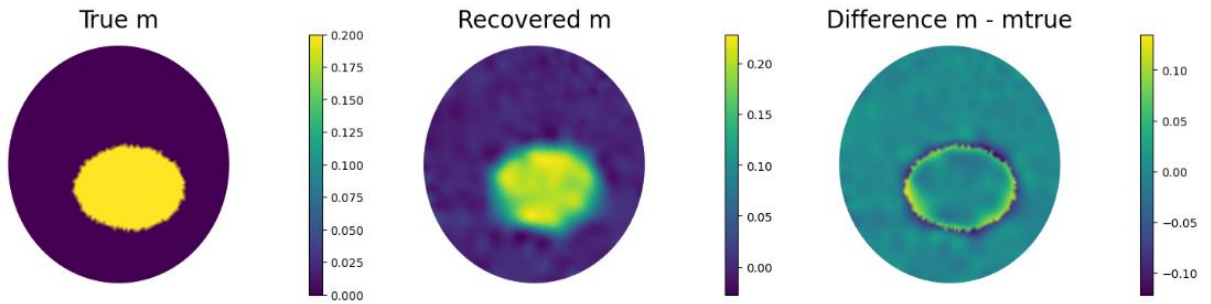
1. Solve the inverse problem for 3 levels of mesh refinement and report the number of Gauss Newton and CG iterations for each case.

First, the optimal regularization is determined (2nd code block in Appendix). The optimal regularization is chosen manually (between 1e-8 and 1e-2) by finding the regularization that best reconstructs the “true” medium properties.

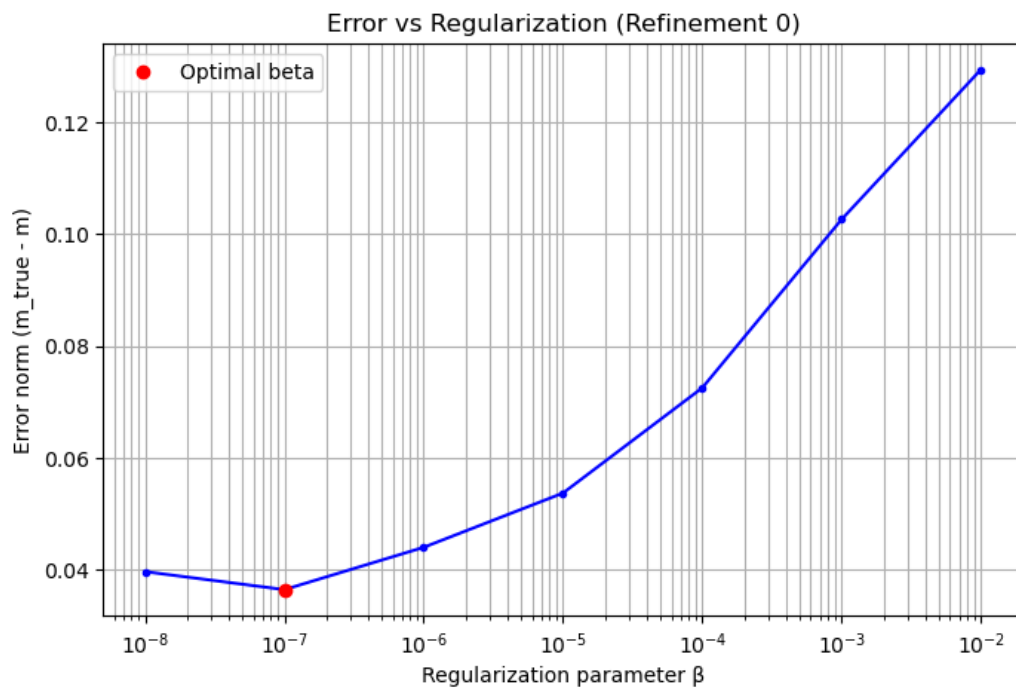
For mesh 1 (coarsest mesh), we notice a regularization of 1e-6 seems ideal, visually:



For a regularization of $1e-7$, we achieve the minimal L2 difference between the true and recovered parameter fields:

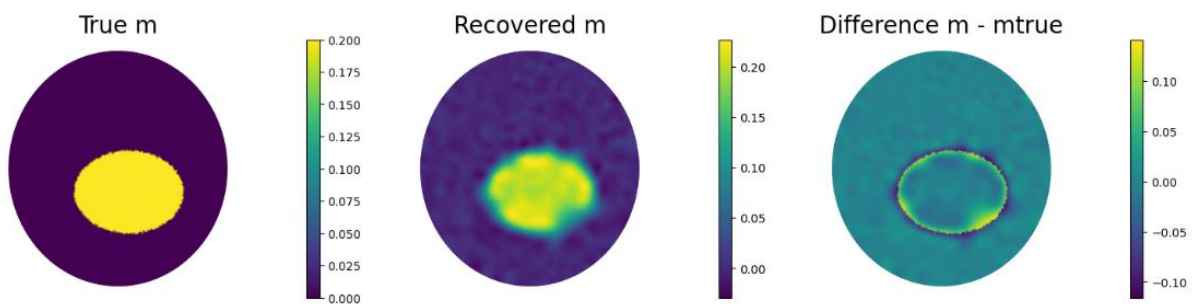


For refinement 0, a regularization of $1e-7$ seems the best (numerically) while visually, $1e-6$ seems better (smoother parameter field).

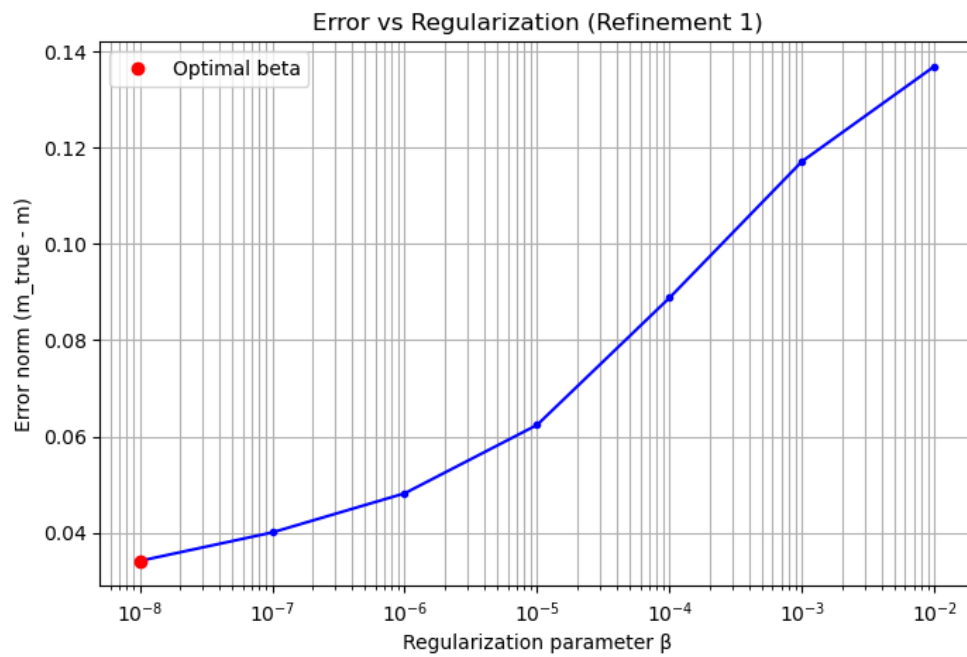
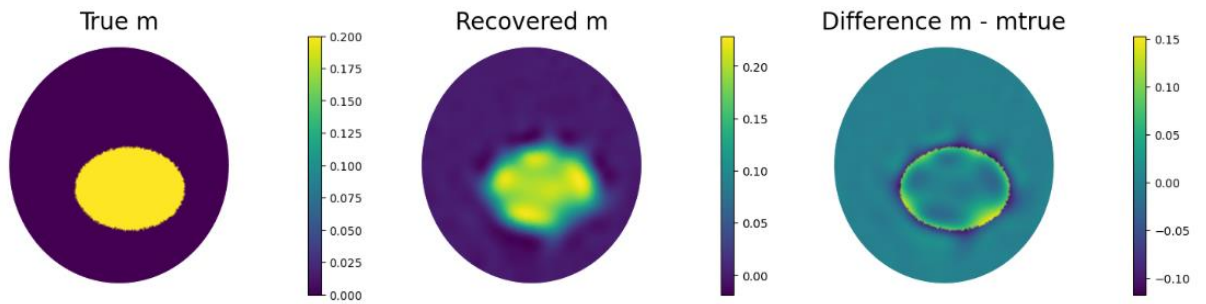


For refinement number 1:

Numerically, the least error is found for a regularization of $1e-8$:

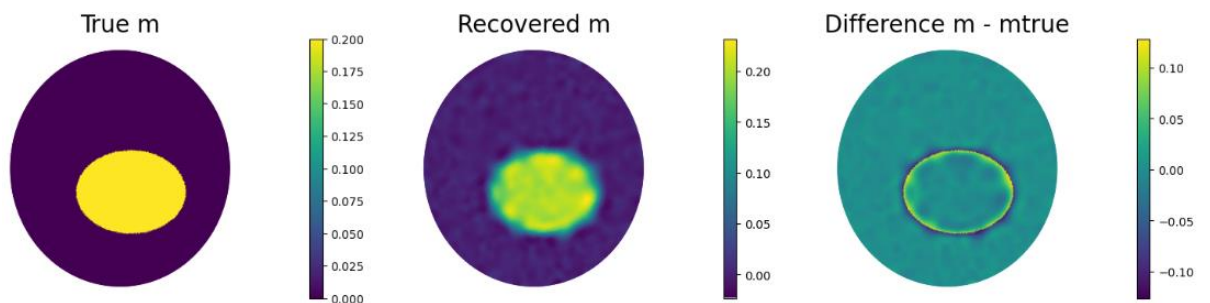


Regularization of $1e-7$ seems ideal, visually:

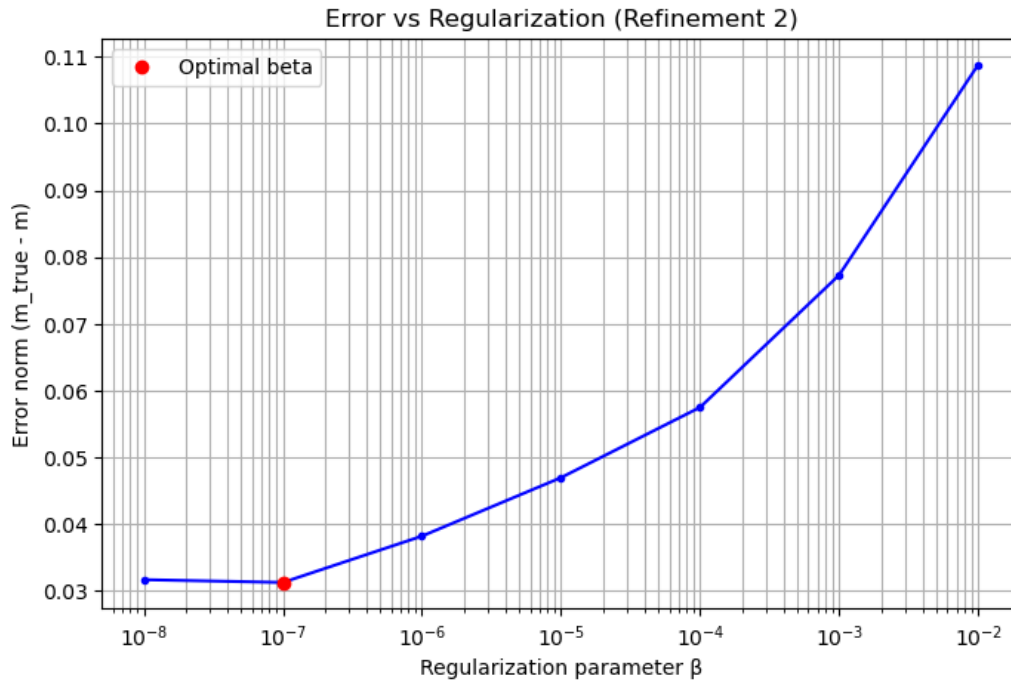
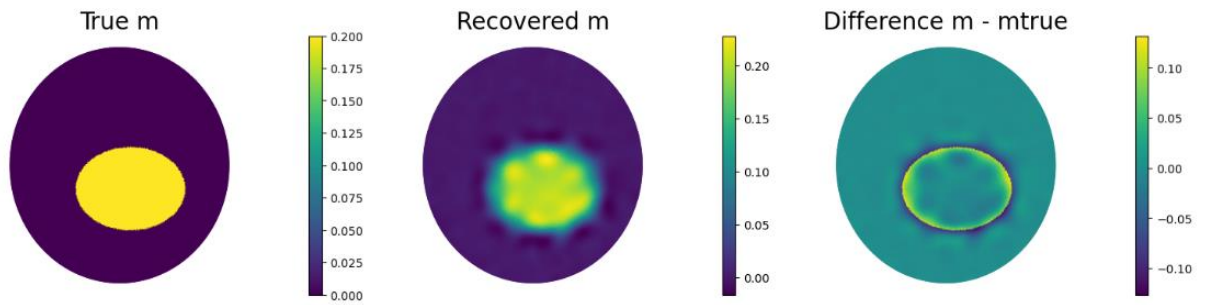


For refinement 2:

Numerically, the least error is found for a regularization of $1e-8$:



Visually, $1e-6$ seems best:



Based on the numerically chosen regularization parameter ($1e-7$ for the coarse mesh, $1e-8$ for refinement 1 and $1e-7$ for refinement 2), we have the following iteration results

```

=== Summary of Results ===

```

Refinement	Mesh Size	Optimal β	Total Iters	GN Iters	Total CG Iters	Final Cost	Final GradNorm	m_diff L2
0	6402	$1.00e-07$	16	9	3211	$4.26e-07$	$1.36e-09$	$3.64e-02$
1	25608	$1.00e-08$	14	9	4002	$1.03e-07$	$7.57e-09$	$3.41e-02$
2	102432	$1.00e-07$	18	9	6202	$2.92e-06$	$8.65e-09$	$3.13e-02$

Increasing the mesh refinement seems to increase the number of CG iterations, while the number of total Newton iterations remains roughly constant. The nonlinearity of the problem, thus, does not change as the mesh is refined. The increase in CG iterations is expected: refining the mesh increases the number of degrees of freedom, which leads to larger and potentially more ill-conditioned linear systems to solve at each Newton step. Consequently, more CG iterations are required to solve the linearized subproblems to the desired tolerance.

For a comparison at the same regularization level, we choose a regularization of $1e-6$:

=== Summary of Results ===

Refinement	Total Iters	GN Iters	Total CG Iters	Final Cost	Final GradNorm	m_diff L2
0	17	9	1595	1.12e-06	5.68e-10	4.04e-02
1	18	9	3572	3.39e-06	6.91e-10	3.64e-02
2	15	9	2258	1.64e-06	2.80e-09	4.03e-02

Again, the number of Newton iterations for convergence remains roughly constant; this is as expected since Newton convergence is independent of parameter/problem dimension.

Meanwhile, CG iteration counts still exhibit a slight increase with refinement, reflecting the increasing size and complexity of the discretised forward/adjoint operators.

Cost function, gradient are same as in the steepest decent code for problem II.

Hessian action:

```
# Weak forms
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# Reassemble beta-dependent forms
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Solve state equation
state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
dl.solve(state_A, u.vector(), state_b)

# Evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# Solve adjoint
adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
dl.solve(adjoint_A, p.vector(), adjoint_RHS)

# Evaluate gradient
MG = dl.assemble(grad_misfit + grad_reg)
dl.solve(M, g, MG)
grad2 = g.inner(MG)
gradnorm = np.sqrt(grad2)

# Set CG tolerance
if iter == 1:
    gradnorm_ini = gradnorm
tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

# Assemble matrices
W_varf = ufl.inner(u_trial, u_test) * ufl.dx
C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

W = dl.assemble(W_varf)
C = dl.assemble(C_varf)
Wum = dl.assemble(Wum_varf)
Wmm = dl.assemble(Wmm_varf)
```

```

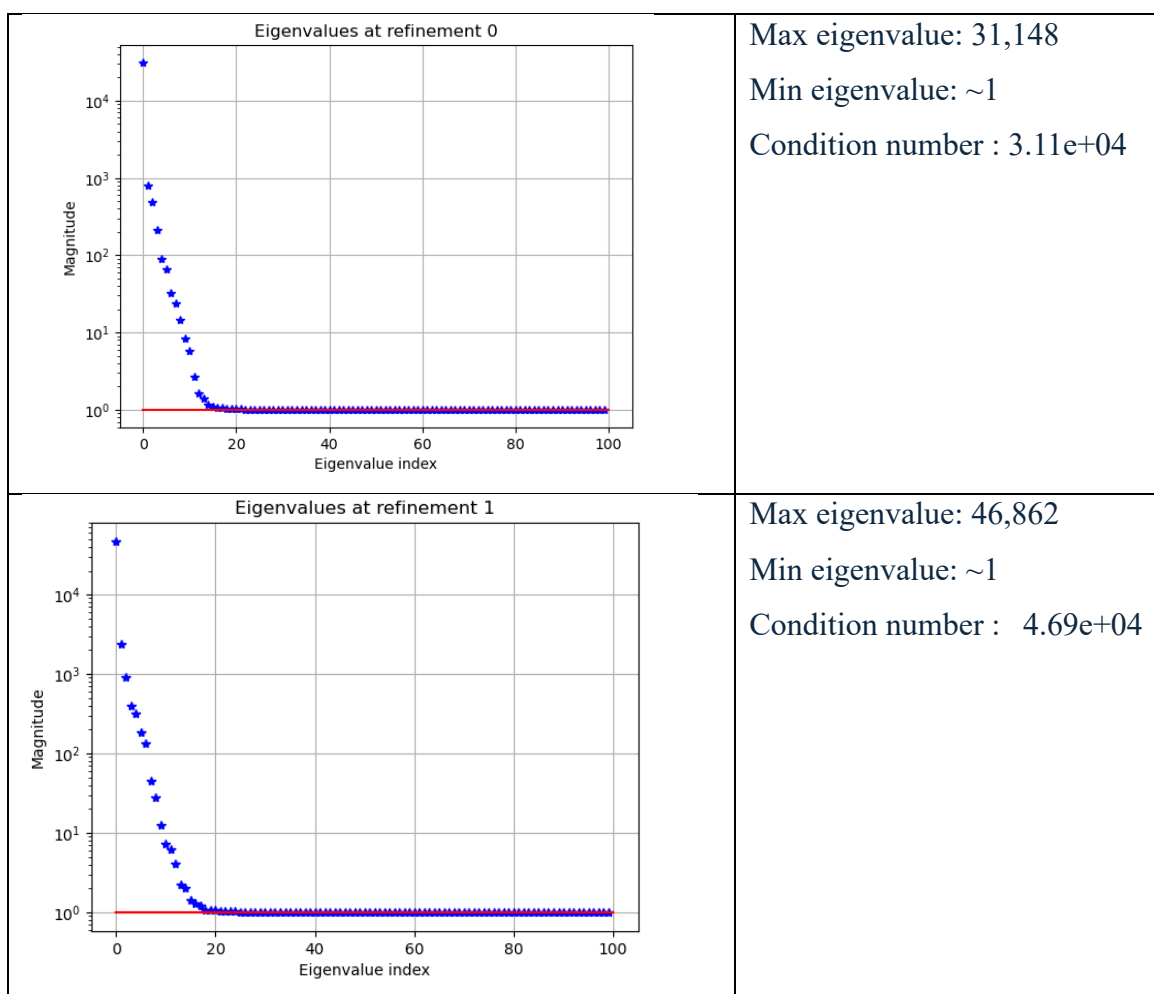
# Define Hessian apply operator
Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
P = R + 0.1*beta * M
Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
Psolver.set_operator(P)

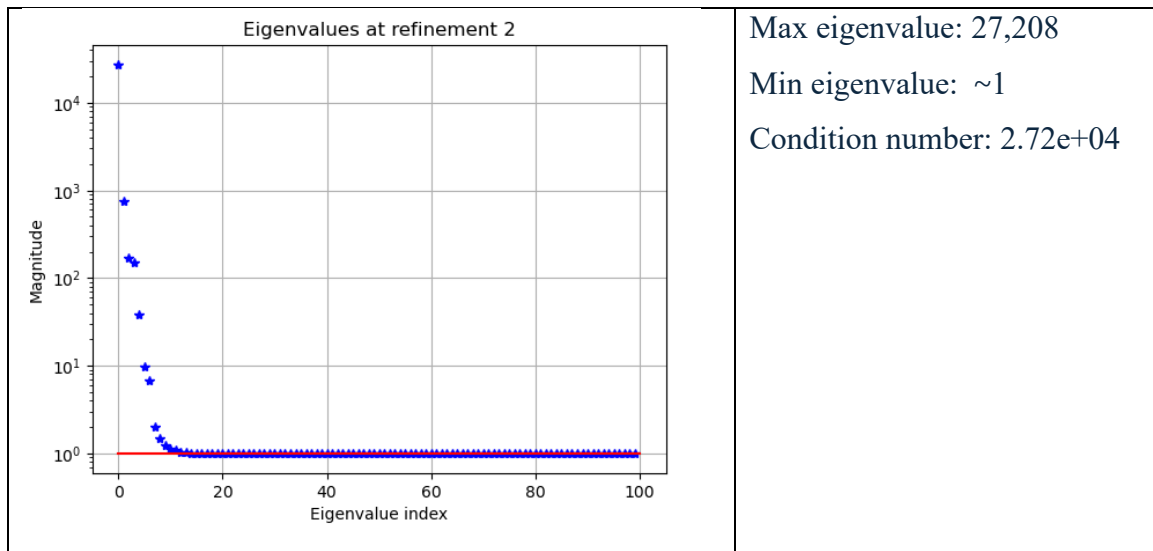
solver = hp.CGSolverSteihaug()
solver.set_operator(Hess_Apply)
solver.set_preconditioner(Psolver)
solver.parameters["rel_tolerance"] = tolcg
solver.parameters["zero_initial_guess"] = True
solver.parameters["print_level"] = -1

```

2. Spectrum of regularization-preconditioned Hessian at solution of inverse problem

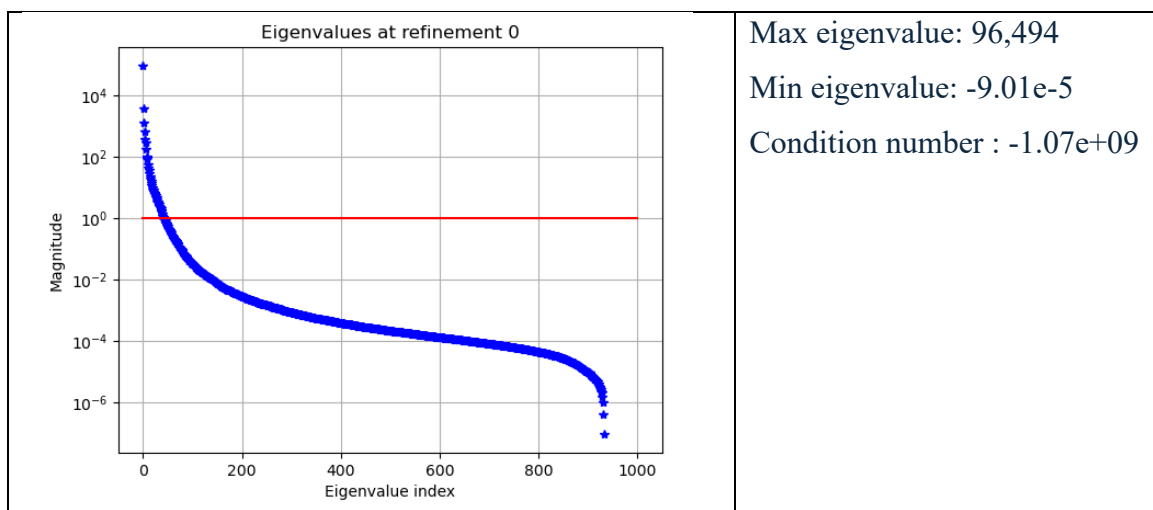
a. As the mesh is refined, the eigenvalues generally become more spread apart:

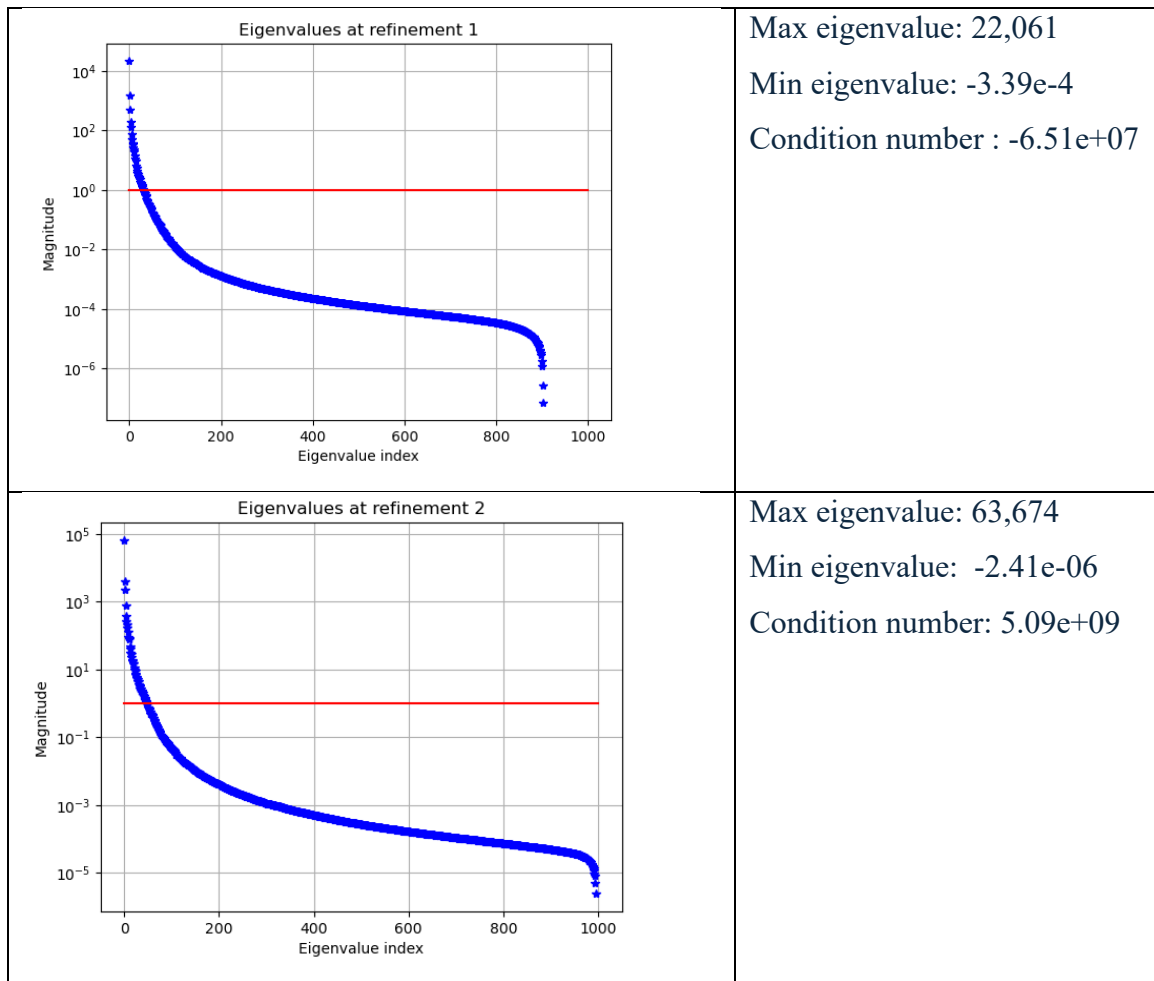




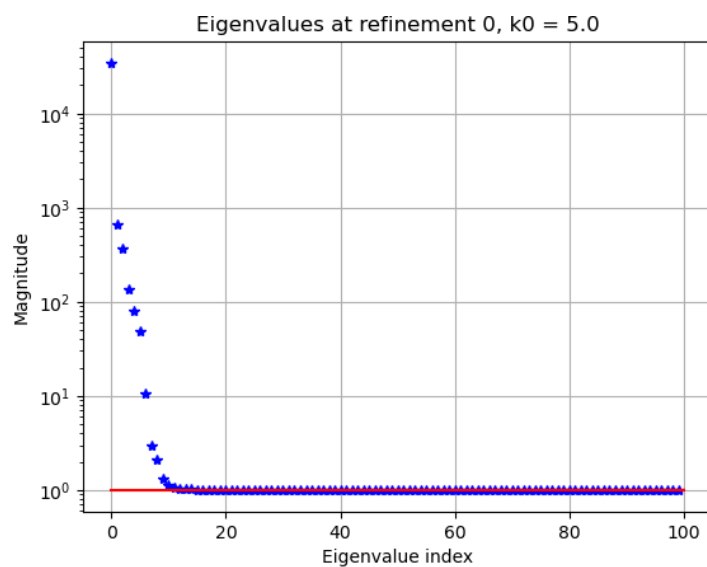
As refinement increases, the eigenvalue range of the Hessian seems to increase. The condition number also seems to generally increase as the mesh is refined, suggesting that the problem becomes less well-posed as refinement increases. With more degrees of freedom/higher dimension, the Hessian can capture more intricacies that are inherent to small scale problems, decreasing the stability of the problem while allowing for more details of the parameter field to be recovered. Notice that only the maximum eigenvalues are increased; the small eigenvalues of magnitude ~ 1 remain constant (these are preconditioned/affected by the regularization).

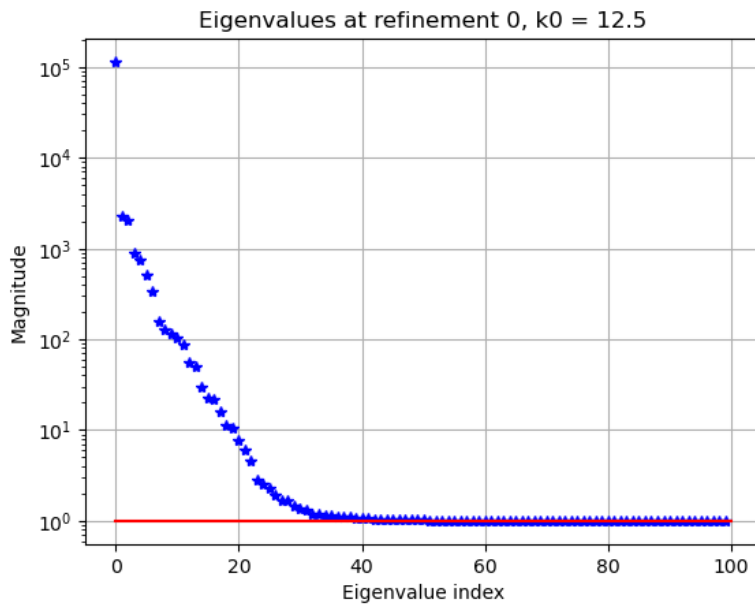
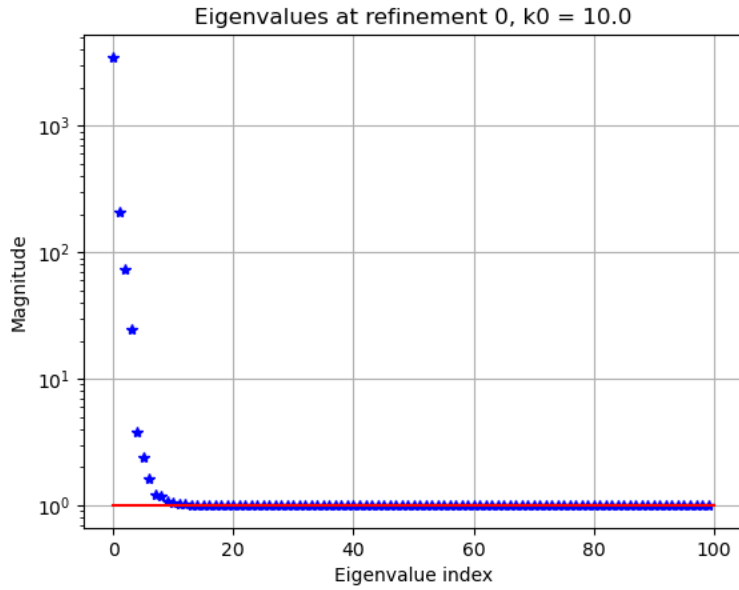
For comparison, the non-regularized precondition hessian eigenvalues are:





- b. For $k_0=5.0$, the max eigenvalue is $\sim 34,332$. For $k_0=10.0$, the max eigenvalue is $\sim 3,488$. For $k_0=12.5$, the max eigenvalue is $\sim 113,784$. The minimum eigenvalues for all of these are 1.





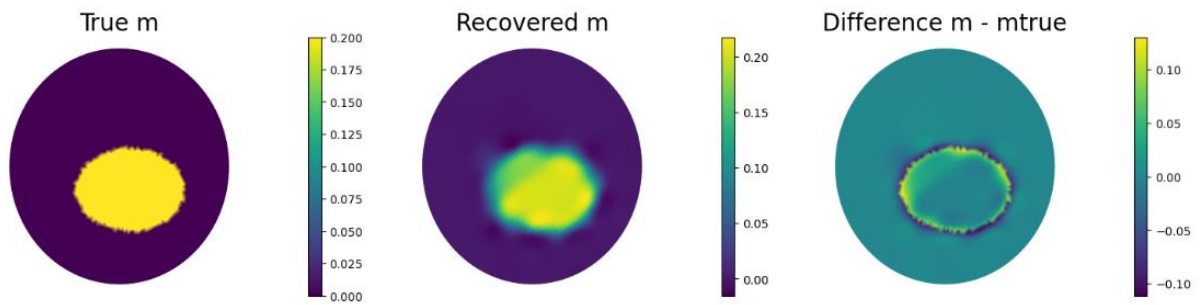
The Hessian matrix is derived from the data misfit and the regularization. The misfit term is influenced by the Helmholtz equation's sensitivity to the parameter m , which is dependent on k_0 . Larger k_0 (higher frequency) leads to higher sensitivity of the solution u to the parameter m , which amplifies the eigenvalues of the misfit part of the Hessian. So, as k_0 increases, the largest eigenvalues (corresponding to the misfit of the Hessian) also grow. Meanwhile, the regularization affects/boosts the smaller eigenvalues. Since we are plotting the regularized-preconditioned Hessian's eigenvalues, the smaller eigenvalues are boosted to be at a minimum of 1, instead of decaying to 0. The condition number seems to increase as the wave number increases, suggesting worse conditioning of the inverse problem.

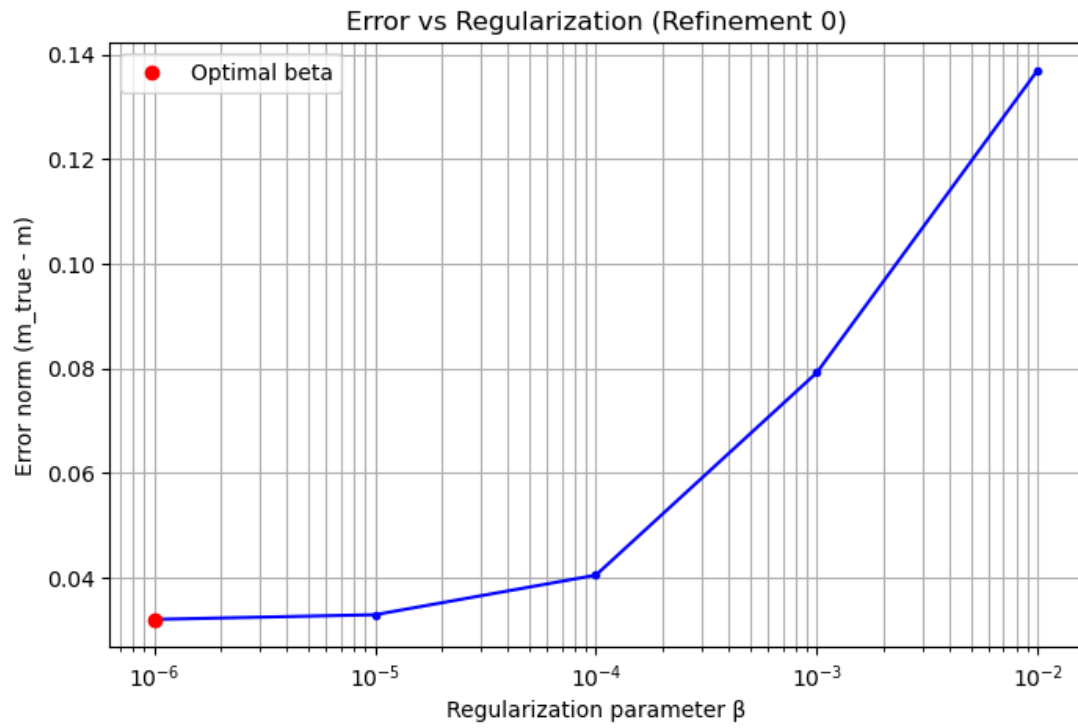
Utilizing frequency hopping to solve the high-frequency Helmholtz problem allows for the number of Newton and CG iterations to remain low (between 100 – 400 CG iterations for about 12 Newton iterations) as the frequency is increased. As the frequency is increased, we observe more instability and slower CG convergence (~ 400 CG iterations compared to the ~ 100 CG iterations for the coarse mesh).

3. With Total Variation regularization, we weaken the constraints for convergence: instead of a $1e-8$ factor reduction in gradient norm, we seek a minimum of $1e-4$ factor reduction in the norm of the gradient (leading to a gradient norm of about $1e-5$). Total variation has a highly non-linear landscape, so even a $1e-4$ relative error is quite good in terms of parameter recovery.

For refinement 0 (coarsest mesh):

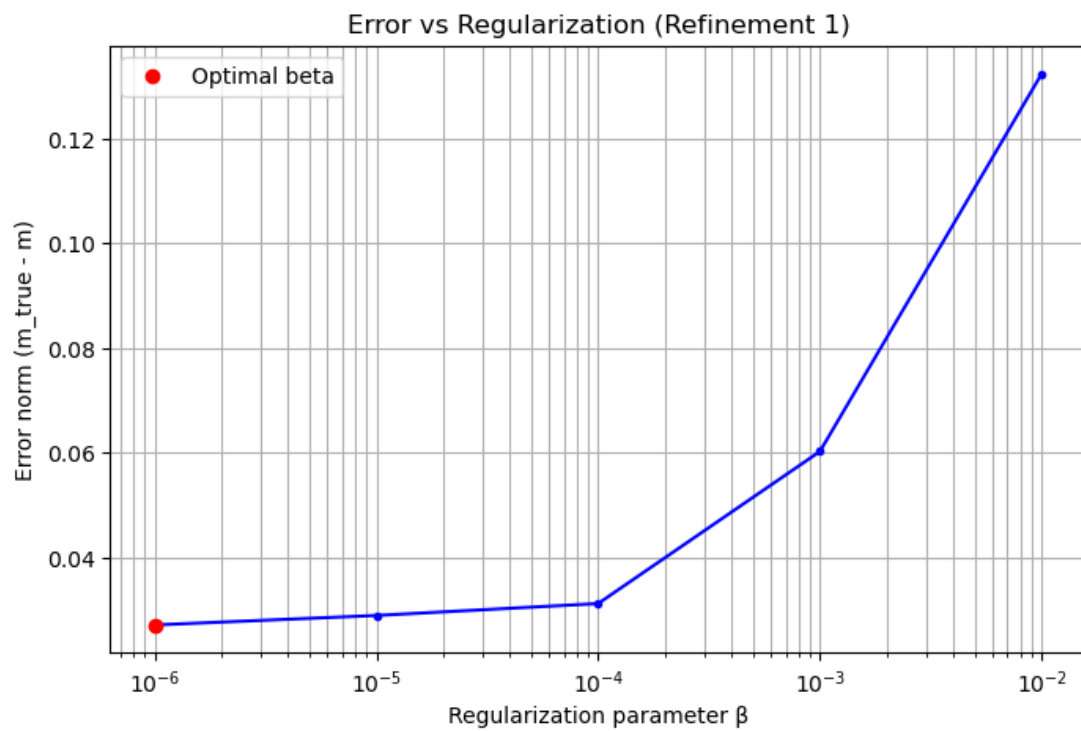
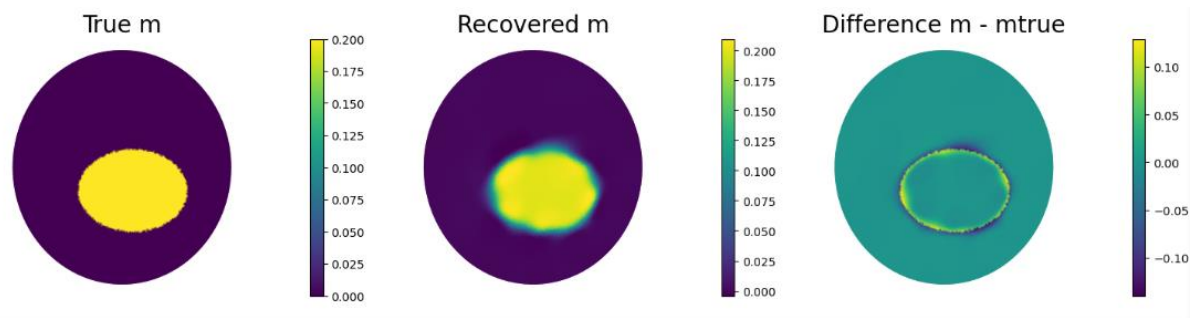
At a regularization of $1e-6$:





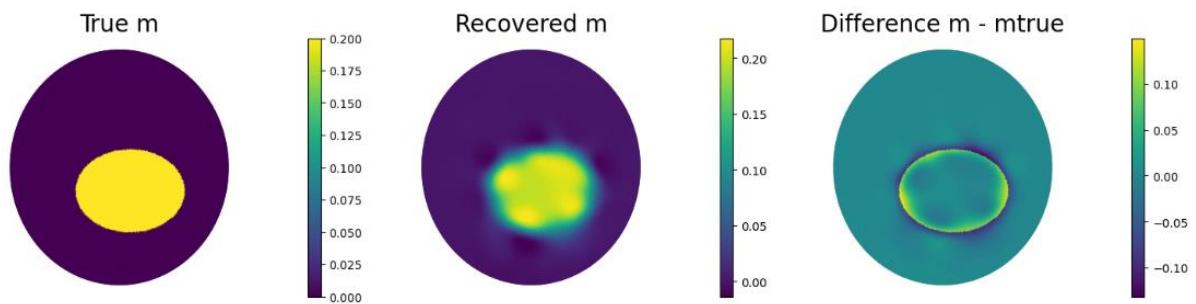
For refinement 1:

At a regularization of $1e-6$:

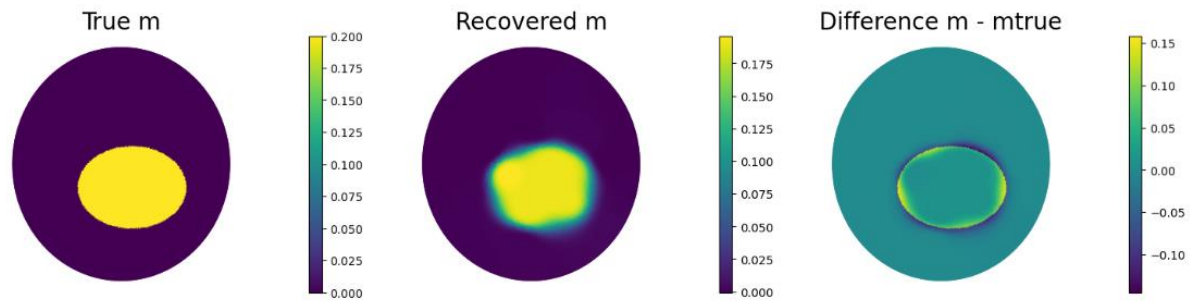


For refinement 2:

At a regularization of $1e-6$:



At a regularization of $1e-5$:



Visually and numerically, a regularization of $1e-5$ seems better at reconstructing the parameter field. Note that due to high runtimes, further values of regularization were not tested.

At the same regularization of $1e-6$, we observe:

Mesh Refinement	Newton Iterations	CG Iterations
0	12	978
1	14	2651
2	14	365

At the ideal regularizations (different regularization for optimal parameter recovery at each refinement):

Mesh Refinement	Regularization (minimal error in parameter recovery)	Newton Iterations	CG Iterations
0	$1e-6$	12	978
1	$1e-6$	14	2651
2	$1e-5$	19	1139

Total variation generally takes more CG iterations for convergence as the mesh is refined.

This is similar to problem 1 (Tikhonov regularization), where the higher problem dimension requires more linear iterations for convergence at each Newton step. Additionally, the number of Newton iterations for convergence remains mostly stable across mesh refinement.

Total variation regularization requires a few more CG iterations for convergence than for Tikhonov regularization. In problem 1 (Tikhonov regularization) and problem 3 (total variation), both methods take between 1000-2000 iterations to converge. However, total variation has a much higher gradient tolerance for converge ($1e-4$ vs. $1e-8$ for the Tikhonov

regularization problem); so, we can conclude that total variation leads to more numerically unstable problems that require large numbers of CG iterations for convergence (more so than for problems using Tikhonov regularization). Notice that one order of magnitude change in regularization (from $1e-6$ to $1e-5$) increases the number of CG iterations by about a factor of 4; this suggests that total variation convergence is highly dependent on regularization tuning (at least more so than for Tikhonov regularization) to suppress fine instabilities/noise while still maintaining crisp/sharp edges in the parameter field.

Cost function/gradient/hessian action:

```
def cost(u, d, m, beta):
    reg = beta * dl.assemble(ufl.sqrt(ufl.inner(ufl.grad(m), ufl.grad(m)) + epsilon)*ufl.dx) # TV regularization
    misfit = 0.5 * dl.assemble((u-d)**2*ufl.dx)
    return [reg + misfit, misfit, reg]
# Gradient of TV term - now correctly formulated
def grad_tv(m, m_test, beta, epsilon):
    grad_m = ufl.grad(m)
    norm_grad_m = ufl.sqrt(ufl.inner(grad_m, grad_m) + epsilon)
    return beta * ufl.inner(grad_m, ufl.grad(m_test)) / norm_grad_m * ufl.dx

# Weak forms (these don't depend on beta)
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
```

```
def get_R_varf(m, m_trial, m_test, beta, epsilon, use_gauss_newton):
    grad_m = ufl.grad(m)
    norm_grad_m = ufl.sqrt(ufl.inner(grad_m, grad_m) + epsilon)

    if use_gauss_newton:
        # Smoothed approximation (simpler form)
        return beta * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) / norm_grad_m * ufl.dx
    else:
        # Exact Hessian of smoothed TV term
        term1 = ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) / norm_grad_m
        term2 = - (ufl.inner(grad_m, ufl.grad(m_trial)) *
                  ufl.inner(grad_m, ufl.grad(m_test))) / (norm_grad_m ** 3)
        return beta * (term1 + term2) * ufl.dx

# Weak forms (these don't depend on beta)
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
```

```
# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)
```

```
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1 - ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx
grad_reg = grad_tv(m, m_test, beta, epsilon)
grad_total = grad_misfit + grad_reg
```

```
# Define other variational forms
W_varf = ufl.inner(u_trial, u_test) * ufl.dx
C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1 - (ufl.tanh(m)**2)), (u_test)) * ufl.dx
Wum_varf = m_trial * p * k0**2 * p_test * (1 - ufl.tanh(m)**2) * ufl.dx
Wmm_varf = - (m_trial * p * k0**2 * (u - u_inc) ** 2 * ufl.tanh(m) * (1 - ufl.tanh(m)**2) * m_test) * ufl.dx

W = dl.assemble(W_varf)
```



```

adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
dl.solve(adjoint_A, p.vector(), adjoint_RHS)

# Evaluate gradient
MG = dl.assemble(grad_misfit + grad_reg)
dl.solve(M, g, MG)
grad2 = g.inner(MG)
gradnorm = np.sqrt(grad2)

# Set CG tolerance
if iter == 1:
    gradnorm_ini = gradnorm
tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

# Assemble matrices
C = dl.assemble(C_varf)
Wum = dl.assemble(Wum_varf)
Wmm = dl.assemble(Wmm_varf)

# Track if we're using Gauss-Newton
use_gauss_newton = (iter < 20)
if use_gauss_newton:
    gauss_newton_iters += 1

# Assemble the appropriate regularization Hessian
R_varf = get_R_varf(m, m_trial, m_test, beta, epsilon, use_gauss_newton)
R = dl.assemble(R_varf)

if use_gauss_newton:
    # Early iterations: simple Laplacian
    P_varf = beta * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
else:
    # Later iterations: weighted Laplacian
    grad_m = ufl.grad(m)
    norm_grad_m = ufl.sqrt(ufl.inner(grad_m, grad_m) + epsilon)
    P_varf = beta * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) / norm_grad_m * ufl.dx

P = dl.assemble(P_varf + 0.1*beta*Mvarf)

# Define Hessian apply operator
Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
# P = R + 0.1*beta * M
Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
Psolver.set_operator(P)

solver = hp.CGSolverSteihaug()
solver.set_operator(Hess_Apply)
solver.set_preconditioner(Psolver)
solver.parameters["rel_tolerance"] = tolcg
solver.parameters["zero_initial_guess"] = True
solver.parameters["print_level"] = -1

# Solve Newton system
solver.solve(m_delta, -MG)

```

IV. Inverse Problem for Burger's Equation

IV) Inverse problem for Burger's equation

14

$$1. \quad \begin{aligned} u_t + uu_x - (mu_x)_x &= f \quad \text{in } (0, L) \times (0, T) \\ u(0, t) = u(L, t) &= 0 \quad \forall t \in [0, T] \\ u(x, 0) &= 0 \quad \forall x \in [0, L] \end{aligned}$$

$$f(m) = \frac{1}{2} \int_0^T \int_0^L (u-d)^2 dx dt + \frac{\beta}{2} \int_0^L \frac{dm}{dx} \frac{dm}{dx} dx$$

Let $p(x, t)$ be a test function of $[0, L] \times [0, T]$ st
 $p(0, t) = p(L, t) = 0 \quad \forall t \in [0, T]$

To get the weak form of the PDE, we first multiply the strong form by the test function $p(x, t)$, then reverse integrate by parts over the space-time domain.

$$\begin{aligned} \int_0^T \int_0^L p(x, t) u_t + p(x, t) uu_x - p(x, t) (mu_x)_x dx dt \\ = \int_0^T \int_0^L f p(x, t) dx dt \end{aligned}$$

$$\Rightarrow \int_0^T \int_0^L p \left(\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \frac{\partial}{\partial x} \left(m \frac{\partial u}{\partial x} \right) - f \right) dx dt = 0$$

$$\int_0^T \int_0^L p \frac{\partial}{\partial x} \left(m \frac{\partial u}{\partial x} \right) dx dt = - \int_0^T \int_0^L m \frac{\partial p}{\partial x} \frac{\partial u}{\partial x} dx dt + \int_0^T \int_{\Gamma} p m \frac{\partial u}{\partial x} \cdot \hat{n} ds dt$$

note in 1D, Γ is simply the 2 ends of the 1D spatial field

$$\begin{aligned} \int_0^T \int_{\Gamma} p m \frac{\partial u}{\partial x} \cdot \hat{n} ds dt &= \int_0^T \left[p m \frac{\partial u}{\partial x} \cdot \hat{n} \right]_{x=0}^{x=L} dt \\ &= \int_0^T \underbrace{p(L, t) \left(m \frac{\partial u}{\partial x} \cdot \hat{n} \right)}_{BC^0} \Big|_{x=L} - \underbrace{p(0, t) \left(m \frac{\partial u}{\partial x} \cdot \hat{n} \right)}_{BC^0} \Big|_{x=0} dt \end{aligned}$$

15

So, the weak form of the PDE is:

$$\int_0^T \int_0^L \left[p \left(\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - f \right) + m \frac{\partial p}{\partial x} \frac{\partial u}{\partial x} \right] dx dt = 0 \quad \forall p \in P$$

P is space for p.

And $u \in U$ s.t. $\mathcal{J}$ is true $\forall p \in P$

the dirichlet BCs of p are: $p(0, t) = p(L, t) = 0$

Definition of parameter, state, adjoint space:

$U: u \in H^1[0, L] \times L^2[0, T]$ $M: m \in H^1[0, L]$

$P: p \in H^1[0, L] \times L^2[0, T]$

16

IV-2) Derive adjoint, gradient (f.w.r.t m)

$$\mathcal{J}(u, m, p) = \frac{1}{2} \int_0^T \int_0^L (u-d)^2 dx dt + \frac{1}{2} \int_0^L \frac{dm}{dx} \frac{dm}{dx} dx$$

$$+ \int_0^T \int_0^L p \left[\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - f \right] + \frac{\partial p}{\partial x} \frac{\partial u}{\partial x} m dx dt$$

• For the adjoint: $\delta_u \mathcal{J} = 0 \quad \forall \hat{u} \in U$

$$\int_0^T \int_0^L (u-d) \hat{u} dx dt + \int_0^T \int_0^L \left[p \left[\frac{\partial \hat{u}}{\partial t} + \hat{u} \frac{\partial u}{\partial x} + u \frac{\partial \hat{u}}{\partial x} \right] + \frac{\partial p}{\partial x} \frac{\partial \hat{u}}{\partial x} m \right] dx dt = 0 \quad \forall \hat{u} \in U$$

weak form of adjoint $\nearrow$

For the strong form, integrate by parts:

$$\int_0^T \int_0^L (u-d) \hat{u} dx dt + \int_0^T \int_0^L \hat{u} \left[-\frac{\partial p}{\partial t} + p \frac{\partial u}{\partial x} - \frac{d}{dx} (p u) \right. \\ \left. - \frac{d}{dx} \left(m \frac{\partial p}{\partial x} \right) \right] dx dt$$

note: m is not necessary in ID

$$+ \int_0^T \left[\hat{u} p \right]_{x=0}^{x=L} dt + \int_0^T \left[m \hat{u} \frac{\partial p}{\partial x} \right]_{x=0}^{x=L} dt + \int_0^L \left[\hat{u} p \right]_0^T dx$$

$= 0 \quad \forall \hat{u} \in U$

$$\int_0^T \int_0^L (u-d) \hat{u} dx dt + \int_0^T \int_0^L \hat{u} \left[-\frac{\partial p}{\partial t} + p \frac{\partial u}{\partial x} - \frac{d}{dx} (p u) \right. \\ \left. - \frac{d}{dx} \left(m \frac{\partial p}{\partial x} \right) \right] dx dt$$

$$+ \int_0^L \left[\hat{u} p \right]_{t=0}^{t=T} dx = 0 \quad \forall \hat{u} \in U$$

$$p \frac{\partial u}{\partial x} - \frac{\partial}{\partial x} (pu) = p \frac{\partial u}{\partial x} - \frac{\partial p}{\partial x} u - \frac{\partial u}{\partial x} p = - \frac{\partial p}{\partial x} u \quad 17$$

$$\int_{T_1}^T \int_0^L \hat{u}(u-d) + \int_0^T \int_0^L \hat{u} \left[-\frac{\partial p}{\partial t} - \frac{\partial p}{\partial x} u - \frac{\partial}{\partial x} \left(m \frac{\partial p}{\partial x} \right) \right] dx dt + \int_0^L [\hat{u} p] \Big|_{t=0}^{t=T} dx = 0 \quad \forall \hat{u} \in \mathcal{U}$$

Since $\hat{u}$ is arbitrary:

note, $\hat{u}(x,0) = 0 \quad \forall x \in [0,L]$
as per u's BC, so, we only
can restrict p at time T

$$\frac{\partial p}{\partial t} + \frac{\partial p}{\partial x} u + \frac{\partial}{\partial x} \left(m \frac{\partial p}{\partial x} \right) = u-d \quad \text{on } (0,L) \times (T_1, T)$$

$$p(0,t) = p(L,t) = 0$$

$$p(x,T) = 0 \quad \forall x \in [0,L]$$

strong form of adjoint formulation $\uparrow$

• For the gradient, $\delta_m \mathcal{L} = 0 \quad \forall \hat{m} \in \mathcal{M}$

$$\beta \int_0^L \frac{dm}{dx} \frac{d\hat{m}}{dx} dx + \int_0^T \int_0^L \frac{\partial p}{\partial x} \frac{\partial u}{\partial x} \hat{m} dx dt = \langle G(m), \hat{m} \rangle$$

$$\forall \hat{m} \in \mathcal{M}$$

weak form of gradient $\uparrow$

Integrate by parts for strong form:

$$\beta \int_0^L \hat{m} \frac{d^2 m}{dx^2} + \int_0^T \int_0^L \hat{m} \left(\frac{\partial p}{\partial x} \cdot \frac{\partial u}{\partial x} \right) dx dt = \langle G(m), \hat{m} \rangle$$

$$+ \beta \hat{m} \frac{dm}{dx} \cdot \vec{n} \Big|_{x=0}^{x=L}$$

again,
 $\vec{n}$ is redundant

since $\hat{m}$ is arbitrary, the gradient in strong form is:

$$-\beta \frac{\partial^2 m}{\partial x^2} + \int_0^T \frac{\partial p}{\partial x} \frac{\partial u}{\partial x} dt \quad \text{at } (x, t) \in (0, L) \times [0, T]$$

$$-\beta \frac{dm}{dx} \Big|_{x=0} \quad \text{at } x=0$$

$$\beta \frac{dm}{dx} \Big|_{x=L} \quad \text{at } x=L$$

Appendix (codes for above problems):

II.1.a

```
import matplotlib.pyplot as plt
%matplotlib inline

import dolfin as dl
import ufl

from hippylib import nb

import numpy as np
import logging

logging.getLogger('FFC').setLevel(logging.WARNING)
logging.getLogger('UFL').setLevel(logging.WARNING)
dl.set_log_active(False)

np.random.seed(seed = 42)
```

```
# create mesh and define function spaces
mesh = dl.Mesh("circle.xml")
print("Refinement level 0; Number of mesh elements: {}".format(mesh.num_cells()))

nrefinements = 0
for i in range(nrefinements):
    mesh = dl.refine(mesh)
    print("Refinement level {}; Number of mesh elements: {}".format(i+1, mesh.num_cells()))

Vm = dl.FunctionSpace(mesh, 'Lagrange', 1)
Vu = dl.FunctionSpace(mesh, 'Lagrange', 2)

# The true parameter
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

# plot
plt.figure(figsize=(15,5))
nb.plot(mesh, subplot_loc=121, mytitle="Mesh", show_axis='on')
nb.plot(mtrue, subplot_loc=122, mytitle="True parameter field")
plt.show()
```

```
# initialize input functions
k0 = 5.
k0squared = dl.Constant(k0*k0)

nsources = 3
theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
radius = 1.05
s0 = radius*np.cos(theta_view)
s1 = radius*np.sin(theta_view)

u_is = []

for i in np.arange(nsources):
    distance_source = dl.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1))",
                                     s0 = s0[i], s1 = s1[i], degree=1)
    u_i_exp = dl.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                             k0=k0, distance_source=distance_source, degree=6)
    u_is.append(dl.interpolate(u_i_exp, Vu) )

# plot
plt.figure(figsize=(15,3.3))
nb.plot(u_is[0], subplot_loc=131, mytitle="First source")
nb.plot(u_is[1], subplot_loc=132, mytitle="Second source")
nb.plot(u_is[2], subplot_loc=133, mytitle="Third source")
plt.show()
```

```

encoding = np.random.normal(size=nsources)

u_inc = dl.Function(Vu)

for i, u_i in enumerate(u_is):
    u_inc.vector().axpy(encoding[i], u_i.vector())

plt.figure(figsize=(7.5,5))
nb.plot(u_inc, mytitle="Super source")
plt.show()

```

```

def pde_varf(u,m,p):
    return ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx \
        -k0squared*(dl.Constant(1.) - ufl.tanh(m) )*u*p*ufl.dx \
        - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx

bc_state = []

```

```

# define parameters for the optimization
tol = 1e-3          # Relative tolerance on the gradient norm
maxiter = 1000      # Maximum number of iterations (we need a lot!)
print_any = 10      # We will only print progress on screen every few iterations
plot_any = 50       # We will plot the current solution every few iterations
c_armijo = 1e-5     # The Armijo constant to ensure sufficient descent
alpha = 1           # The initial step-size

# initialize iter counters
iter = 0
converged = False

# initializations
g = dl.Vector()
M.init_vector(g,0)

m_prev = dl.Function(Vm)

#Select whether to use the L^2 inner product (mass matrix M)
# or to precondition steepest descent using the H^1 inner product (stiffness matrix K)
P = M

print( "Nit  cost      misfit      reg      ||grad||      alpha  N backtrack" )

```

```

# noise level
noise_level = 0.01

u_trial = dl.TrialFunction(Vu)
u_test = dl.TestFunction(Vu)

# weak form for setting up the synthetic observations
a_true = ufl.lhs(pde_varf(u_trial,mtrue,u_test))
L_true = ufl.rhs(pde_varf(u_trial,mtrue,u_test))

# solve the forward/state problem to generate synthetic observations
A_true, b_true = dl.assemble_system(a_true, L_true, bc_state)

utru = dl.Function(Vu)
dl.solve(A_true, utru.vector(), b_true)

d = dl.Function(Vu)
d.assign(utru)

# perturb state solution and create synthetic measurements d
# d = u + ||u||/SNR * random.normal
MAX = d.vector().norm("linf")
noise = dl.Vector()
A_true.init_vector(noise,1)
noise.set_local( noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local())) )

d.vector().axpy(1., noise)

# plot
nb.multil_plot([utru, d], ["State solution with mtrue", "Synthetic observations"])
plt.show()

```

```

u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)
# Cost function
beta = 1e-5 # Regularization parameter

# Define cost function
def cost(u, d, m, beta):
    reg = 0.5*beta * dl.assemble( ufl.inner(dl.grad(m), ufl.grad(m))*ufl.dx )
    misfit = 0.5 * dl.assemble( (u-d)**2*ufl.dx )
    return [reg + misfit, misfit, reg]

```



```

while iter < maxiter and not converged:

    # solve the adjoint problem
    adj_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adj_A, p.vector(), adjoint_RHS)

    # evaluate the gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(P, g, MG)

    # calculate the norm of the gradient
    grad_norm2 = g.inner(MG)
    gradnorm = np.sqrt(grad_norm2)

    if iter == 0:
        gradnorm0 = gradnorm

    # linesearch
    it_backtrack = 0
    m_prev.assign(m)
    backtrack_converged = False
    for it_backtrack in range(20):

        m.vector().axpy(-alpha, g )

        # solve the forward problem
        state_A, state_b = dl.assemble_system(a_state, L_state)
        dl.solve(state_A, u.vector(), state_b)

        # evaluate cost
        [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

        # check if Armijo conditions are satisfied
        if cost_new < cost_old - alpha * c_armijo * grad_norm2:
            cost_old = cost_new
            backtrack_converged = True
            break
        else:
            alpha *= 0.5
            m.assign(m_prev) # reset m

    if backtrack_converged == False:
        print( "Backtracking failed. A sufficient descent direction was not found" )
        converged = False
        break

    sp = ""
    if iter == 0:
        m_min, m_max = m.vector().min(), m.vector().max()
        u_min, u_max = u.vector().min(), u.vector().max()
        p_min, p_max = p.vector().min(), p.vector().max()

# weak form for setting up the state equation
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx

# weak form for setting up the adjoint equations
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx

# weak form for gradient
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(ufl.tanh(m))) * (u - u_inc)) * ufl.dx
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# matrix in parameter space
Kvarf = ufl.inner(m_trial, m_test) * ufl.dx + ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
K = dl.assemble(Kvarf)

```

```

if (iter % print_any) == 0 :
    print( "%3d %1s %8.5e %1s %8.5e %1s %8.5e %1s %8.5e %1s %3d" % \
           (iter, sp, cost_new, sp, misfit_new, sp, reg_new, sp, \
            gradnorm, sp, alpha, sp, it_backtrack) )

if (iter % plot_any) == 0 :
    nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
# if (iter % plot_any) == 0:
#     plot_with_fixed_colorbar(
#         [m, u, p],
#         ["m", "u", "p"],
#         # [m_min, u_min, p_min], # vmin for each field
#         # [m_max, u_max, p_max] # vmax for each field
#     )
    plt.show()

# check for convergence
if gradnorm < tol*gradnorm0 and iter > 0:
    converged = True
    print ("Steepest descent converged in ",iter," iterations")

alpha *= 1.5 # Try to increase the step size for the next iteration
iter += 1

if not converged:
    print ( "Steepest descent did not converge in ", maxiter, " iterations")

```

```

m0 = dl.Constant(0 )

n_eps = 32
eps = np.power(2., -np.arange(n_eps))
err_grad = np.zeros(n_eps)

m.assign(m0)

#Solve the fwd problem and evaluate the cost functional
A, state_b = dl.assemble_system (a_state, L_state)
dl.solve(A, u.vector(), state_b)

c0, _, _ = cost(u, d, m, beta)

# Solve the adjoint problem and evaluate the gradient
adj_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
dl.solve(adj_A, p.vector(), adjoint_RHS)

# evaluate the gradient
grad0 = dl.assemble(grad_misfit + grad_reg)

# Define an arbitrary direction m_hat to perform the check
mtilde = dl.Function(Vm).vector()
mtilde.set_local(np.random.randn(Vm.dim()))
mtilde.apply("")
mtilde_grad0 = grad0.inner(mtilde)

for i in range(n_eps):
    m.assign(m0)
    m.vector().axpy(eps[i], mtilde)

    A, state_b = dl.assemble_system (a_state, L_state)
    dl.solve(A, u.vector(), state_b)

    cplus, _, _ = cost(u, d, m, beta)

    err_grad[i] = abs( (cplus - c0)/eps[i] - mtilde_grad0 )

plt.figure()
plt.loglog(eps, err_grad, "-ob", label="Error Grad")
plt.loglog(eps, (.5*err_grad[0]/eps[0])*eps, "-.k", label="First Order")
plt.title("Finite difference check of the first variation (gradient)")
plt.xlabel("eps")
plt.ylabel("Error grad")
plt.legend(loc = "upper left")
plt.show()

```

```

m.assign(m0)

# solve state equation
A, state_b = dl.assemble_system (a_state, L_state)
dl.solve(A, u.vector(), state_b)

# evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# plot
plt.figure(figsize=(15,5))
nb.plot(m,subplot_loc=121, mytitle="m0", vmin=mtrue.vector().min(), vmax=mtrue.vector().max())
nb.plot(u,subplot_loc=122, mytitle="u(m0)")
plt.show()

```

```
l2_norm_squared = d1.assemble((m_diff**2)*d1.dx)
print("Tikhonov, L2-inner product, k0 = 5, ||m - m_true||^2 =", l2_norm_squared)
```

II.1.b

Same code as in II.1.a, until the setup for descent/optimization loop. The only change is $P = K$, rather than $P = M$:

```
# define parameters for the optimization
tol = 1e-3          # Relative tolerance on the gradient norm
maxiter = 1000      # Maximum number of iterations (we need a lot!)
print_any = 10      # We will only print progress on screen every few iterations
plot_any = 50       # We will plot the current solution every few iterations
c_armijo = 1e-5     # The Armijo constant to ensure sufficient descent
alpha = 1           # The initial step-size

# initialize iter counters
iter = 0
converged = False

# initializations
g = d1.Vector()
M.init_vector(g,0)

m_prev = d1.Function(Vm)

#Select whether to use the L^2 inner product (mass matrix M)
# or to precondition steepest descent using the H^1 inner product (stiffness matrix K)
P = K

print( "Nit   cost       misfit       reg       ||grad||       alpha  N backtrack" )
```

II.1.c

Exact same code as in II.1.a, except that $k_0=10$ (assigned during source setup, after mesh setup):

```
# initialize input functions
k0 = 10.
k0squared = d1.Constant(k0*k0)

nsources = 3
theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
radius = 1.05
s0 = radius*np.cos(theta_view)
s1 = radius*np.sin(theta_view)

u_is = []

for i in np.arange(nsources):
    distance_source = d1.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1) )",
                                     s0 = s0[i], s1 = s1[i], degree=1)
    u_i_exp = d1.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                             k0=k0, distance_source=distance_source, degree=6)
    u_is.append(d1.interpolate(u_i_exp, Vu) )

# plot
plt.figure(figsize=(15,3.3))
nb.plot(u_is[0], subplot_loc=131, mytitle="First source")
nb.plot(u_is[1], subplot_loc=132, mytitle="Second source")
nb.plot(u_is[2], subplot_loc=133, mytitle="Third source")
plt.show()
```

```

while iter < maxiter and not converged:

    # solve the adjoint problem
    adj_A, adjoint_RHS = d1.assemble_system(a_adj, L_adj)
    d1.solve(adj_A, p.vector(), adjoint_RHS)

    # evaluate the gradient
    MG = d1.assemble(grad_misfit + grad_reg)
    d1.solve(P, g, MG)

    # calculate the norm of the gradient
    grad_norm2 = g.inner(MG)
    gradnorm = np.sqrt(grad_norm2)

    if iter == 0:
        gradnorm0 = gradnorm

    # Linesearch
    it_backtrack = 0
    m_prev.assign(m)
    backtrack_converged = False
    for it_backtrack in range(20):

        m.vector().axpy(-alpha, g )

        # solve the forward problem
        state_A, state_b = d1.assemble_system(a_state, L_state)
        d1.solve(state_A, u.vector(), state_b)

        # evaluate cost
        [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

        # check if Armijo conditions are satisfied
        if cost_new < cost_old - alpha * c_armijo * grad_norm2:
            cost_old = cost_new
            backtrack_converged = True
            break
        else:
            alpha *= 0.5
            m.assign(m_prev) # reset m

    if backtrack_converged == False:
        print( "Backtracking failed. A sufficient descent direction was not found" )
        converged = False
        break

```

```

sp = ""
if (iter % print_any) == 0 :
    print( "%3d %1s %8.5e %1s %8.5e %1s %8.5e %1s %8.5e %1s %3d" % \
           (iter, sp, cost_new, sp, misfit_new, sp, reg_new, sp, \
            gradnorm, sp, alpha, sp, it_backtrack) )

if (iter % plot_any) == 0 :
    nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
# if (iter % plot_any) == 0:
#     plot_with_fixed_colorbar(
#         [m, u, p],
#         ["m", "u", "p"],
#         [m_min, u_min, p_min], # vmin for each field
#         [m_max, u_max, p_max] # vmax for each field
#     )
    plt.show()

# check for convergence
if gradnorm < tol*gradnorm0 and iter > 0:
    converged = True
    print ("Steepest descent converged in ",iter," iterations")

alpha *= 1.5 # Try to increase the step size for the next iteration
iter += 1

if not converged:
    print ( "Steepest descent did not converge in ", maxiter, " iterations")

```

```

m_diff = d1.Function(Vm)
m_diff.vector()[:] = mtrue.vector() - m.vector()

plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
plt.show()

```

```

l2_norm_squared = d1.assemble((m_diff**2)*d1.dx)
print("Tikhonov, H1-inner product, k0 = 5, ||m - m_true||^2 =", l2_norm_squared)

```


II.2

Same mesh setup as in II.1.a, then, k_0 is reset to 5 and regularization is changed to Total Variation.

```
# reset k0 to standard value
k0 = 5.
k0squared = dl.Constant(k0 * k0)

delta = .01

# weak form for setting up the state equation
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
l_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx

# weak form for setting up the adjoint equations
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
l_adj = - ufl.inner(u - d, p_test) * ufl.dx

# weak form for gradient
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1 - ufl.tanh(ufl.tanh(m))) * (u - u_inc)) * ufl.dx
grad_reg = (dl.Constant(beta) * ufl.inner(ufl.grad(m_test), ufl.grad(m)) / ufl.sqrt(ufl.inner(ufl.grad(m), ufl.grad(m)) + dl.Constant(delta))) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# matrix in parameter space
Kvarf = ufl.inner(m_trial, m_test) * ufl.dx + ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
K = dl.assemble(Kvarf)

m.assign(m0)

# solve state equation
A, state_b = dl.assemble_system(a_state, l_state)
dl.solve(A, u.vector(), state_b)

# evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# plot
plt.figure(figsize=(15,5))
nb.plot(m, subplot_loc=121, mytitle="m0", vmin=mtrue.vector().min(), vmax=mtrue.vector().max())
nb.plot(u, subplot_loc=122, mytitle="u(m0)")
plt.show()
```

Same descent loop as in II.1.a, except max iterations increased to 2000:

```

# define parameters for the optimization
tol = 1e-3      # Relative tolerance on the gradient norm
maxiter = 2000  # Maximum number of iterations (we need a lot!)
print_any = 10  # We will only print progress on screen every few iterations
plot_any = 50   # We will plot the current solution every few iterations
c_armijo = 1e-5 # The Armijo constant to ensure sufficient descent
alpha = 1       # The initial step-size

# initialize iter counters
iter = 0
converged = False

# initializations
g = dl.Vector()
M.init_vector(g,0)

m_prev = dl.Function(Vm)

#Select whether to use the L^2 inner product (mass matrix M)
# or to precondition steepest descent using the H^1 inner product (stiffness matrix K)
P = M

print( "Nit  cost      misfit      reg      ||grad||      alpha  N backtrack" )

while iter < maxiter and not converged:

    # solve the adjoint problem
    adj_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adj_A, p.vector(), adjoint_RHS)

    # evaluate the gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(P, g, MG)

    # calculate the norm of the gradient
    grad_norm2 = g.inner(MG)
    gradnorm = np.sqrt(grad_norm2)

    if iter == 0:
        gradnorm0 = gradnorm

    # linesearch
    it_backtrack = 0
    m_prev.assign(m)
    backtrack_converged = False
    for it_backtrack in range(20):

        m.vector().axpy(-alpha, g )

```

```

# solve the forward problem
state_A, state_b = d1.assemble_system(a_state, l_state)
d1.solve(state_A, u.vector(), state_b)

# evaluate cost
[cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

# check if Armijo conditions are satisfied
if cost_new < cost_old - alpha * c_armijo * grad_norm2:
    cost_old = cost_new
    backtrack_converged = True
    break
else:
    alpha *= 0.5
    m.assign(m_prev) # reset m

if backtrack_converged == False:
    print( "Backtracking failed. A sufficient descent direction was not found" )
    converged = False
    break

sp = ""
if (iter % print_any) == 0 :
    print( "%3d %1s %8.5e %1s %8.5e %1s %8.5e %1s %8.5e %1s %8.5e %1s %3d" % \
        (iter, sp, cost_new, sp, misfit_new, sp, reg_new, sp, \
        gradnorm, sp, alpha, sp, it_backtrack) )

if (iter % plot_any) == 0 :
    nb.multit_plot([m,u,p], ["m","u","p"], same_colorbar=False)
    # if (iter % plot_any) == 0:
    #     plot_with_fixed_colorbar(
    #         [m, u, p],
    #         ["m", "u", "p"],
    #         [m_min, u_min, p_min], # vmin for each field
    #         [m_max, u_max, p_max] # vmax for each field
    #     )
    plt.show()

# check for convergence
if gradnorm < tol*gradnorm0 and iter > 0:
    converged = True
    print ("Steepest descent converged in ",iter," iterations")

alpha *= 1.5 # Try to increase the step size for the next iteration
iter += 1

```

```

if not converged:
    print ( "Steepest descent did not converge in ", maxiter, " iterations")

```

```

l2_norm_squared = d1.assemble((m_diff**2)*d1.dx)
print("Total Variation, L2-inner product, k0 = 5, ||m - m_true||^2 =", l2_norm_squared)

```

k0 is changed to 10, and the process is re-run.

II.3

```

import matplotlib.pyplot as plt
%matplotlib inline

import dolfin as dl
import ufl

from hippylib import nb

import numpy as np
import logging

# Set Logging Levels
logging.getLogger('FFC').setLevel(logging.WARNING)
logging.getLogger('UFL').setLevel(logging.WARNING)
dl.set_log_active(False)

# Set random seed for reproducibility
np.random.seed(seed=42)

# =====
# Mesh and Function Space Setup
# =====
# Create mesh and define function spaces
mesh = dl.Mesh("circle.xml")
print("Refinement level 0; Number of mesh elements: {}".format(mesh.num_cells()))

nrefinements = 0
for i in range(nrefinements):
    mesh = dl.refine(mesh)
    print("Refinement level {}; Number of mesh elements: {}".format(i+1, mesh.num_cells()))

# Define function spaces
Vm = dl.FunctionSpace(mesh, 'Lagrange', 1) # Parameter space
Vu = dl.FunctionSpace(mesh, 'Lagrange', 2) # State/adjoint space

u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)

# =====
# True Parameter Field
# =====
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

# Plot true parameter field and mesh
plt.figure(figsize=(15,5))
nb.plot(mesh, subplot_loc=121, mytitle="Mesh", show_axis='on')
nb.plot(mtrue, subplot_loc=122, mytitle="True parameter field")
plt.show()

# =====
# Source Setup (Multiple Independent Sources)
# =====
k0 = 10.0 # Wavenumber
k0squared = dl.Constant(k0*k0)

nsources = 3
theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
radius = 1.05
s0 = radius*np.cos(theta_view)
s1 = radius*np.sin(theta_view)

u_is = [] # List to store all source fields

for i in np.arange(nsources):
    distance_source = dl.Expression(
        "std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1))",
        s0=s0[i], s1=s1[i], degree=1
    )
    u_i_exp = dl.Expression(
        "-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
        k0=k0, distance_source=distance_source, degree=6
    )
    u_is.append(dl.interpolate(u_i_exp, Vu))

# Plot all sources
plt.figure(figsize=(15,3.3))
for i in range(nsources):
    nb.plot(u_is[i], subplot_loc=131+i, mytitle=f"Source {i+1}")
plt.show()

```



```

# =====
# PDE Variational Form
# =====
def pde_varf(u, m, p, u_inc):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(d1.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# =====
# Generate Synthetic Observations for Each Source
# =====
noise_level = 0.01 # Noise Level
bc_state = [] # Boundary conditions (empty List means no BCs)

u_trial = d1.TrialFunction(Vu)
u_test = d1.TestFunction(Vu)

d_list = [] # List to store observations
utruelist = [] # List to store true states

for i in range(nsources):
    # Solve forward problem with true parameter and current source
    a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test, u_is[i]))
    L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test, u_is[i]))

    A_true, b_true = d1.assemble_system(a_true, L_true, bc_state)

    utruelist = d1.Function(Vu)
    d1.solve(A_true, utruelist.vector(), b_true)

    d_i = d1.Function(Vu)
    d_i.assign(utruelist)

    # Add noise to observations
    MAX = d_i.vector().norm("linf")
    noise = d1.Vector()
    A_true.init_vector(noise, 1)
    noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d_i.vector().get_local())))
    d_i.vector().axpy(1., noise)

    utruelist.append(utruelist)
    d_list.append(d_i)

# Plot observations from each source
plt.figure(figsize=(15,5))
for i in range(nsources):
    plt.subplot(1, nsources, i+1)
    nb.plot(d_list[i], mytitle=f"Observation from source {i+1}")
plt.show()

# =====
# Inverse Problem Setup
# =====
beta = 1e-5 # Regularization parameter

# Define cost function for multiple sources
def cost_multisource(u_list, d_list, m, beta):
    misfit = 0
    for u, d in zip(u_list, d_list):
        misfit += 0.5 * d1.assemble((u-d)**2*ufl.dx)
    reg = 0.5*beta * d1.assemble(ufl.inner(d1.grad(m), ufl.grad(m))*ufl.dx)
    return [reg + misfit, misfit, reg]

# Initialize variables
u_list = [d1.Function(Vu) for _ in range(nsources)] # State variables
p_list = [d1.Function(Vu) for _ in range(nsources)] # Adjoint variables
m = d1.Function(Vm) # Parameter
m.assign(d1.Constant(0.0)) # Initial guess

# Define variational forms for state, adjoint, and gradient
m_test = d1.TestFunction(Vm)
m_trial = d1.TrialFunction(Vm)

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = d1.assemble(Mvarf)

# Stiffness matrix in parameter space (for preconditioning)
Kvarf = ufl.inner(m_trial, m_test) * ufl.dx + ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
K = d1.assemble(Kvarf)

```

```

# =====
# Optimization Loop
# =====
# Optimization parameters
tol = 1e-3      # Relative tolerance on gradient norm
maxiter = 1000  # Maximum iterations
print_any = 10  # Print progress every N iterations
plot_any = 100  # Plot solution every N iterations
c_armijo = 1e-5 # Armijo condition constant
alpha = 1.0     # Initial step size

# Initialize variables for optimization
g = dl.Vector() # Gradient vector
M.init_vector(g, 0)
m_prev = dl.Function(Vm)

# Use stiffness matrix for preconditioning
P = K

print("Nit cost      misfit      reg      ||grad||      alpha N backtrack")

iter = 0
converged = False

while iter < maxiter and not converged:
    # Solve all state equations
    for i in range(nsources):
        a_state = ufl.lhs(pde_varf(u_trial, m, u_test, u_is[i]))
        L_state = ufl.rhs(pde_varf(u_trial, m, u_test, u_is[i]))

        state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
        dl.solve(state_A, u_list[i].vector(), state_b)

    # Evaluate cost
    cost_old, misfit_old, reg_old = cost_multisource(u_list, d_list, m, beta)

    # Solve all adjoint equations and accumulate gradient
    MG_total = dl.Vector()
    M.init_vector(MG_total, 0)
    MG_total.zero()

    for i in range(nsources):
        a_adj = ufl.lhs(pde_varf(p_trial, m, p_test, u_is[i])) # Same form as state
        L_adj = -ufl.inner(u_list[i] - d_list[i], p_test) * ufl.dx

        adj_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj, bc_state)
        dl.solve(adj_A, p_list[i].vector(), adjoint_RHS)

        # Gradient contribution from this source
        grad_misfit_i = (ufl.inner(m_test, p_list[i]) * k0**2 *
                        (1 - ufl.tanh(m)**2) * (u_list[i] - u_is[i])) * ufl.dx
        MG_total.axpy(1.0, dl.assemble(grad_misfit_i))

    # Add regularization term to gradient
    grad_reg = dl.assemble(beta * ufl.inner(ufl.grad(m), ufl.grad(m_test)) * ufl.dx)
    MG_total.axpy(1.0, grad_reg)

    # Precondition the gradient
    dl.solve(P, g, MG_total)

    # Compute gradient norm
    grad_norm2 = g.inner(MG_total)
    gradnorm = np.sqrt(grad_norm2)

    if iter == 0:
        gradnorm0 = gradnorm

```

```

# Line search
it_backtrack = 0
m_prev.assign(m)
backtrack_converged = False

for it_backtrack in range(20):
    m.vector().axpy(-alpha, g)

    # Solve state equations with new m
    for i in range(nsources):
        a_state = ufl.lhs(pde_varf(u_trial, m, u_test, u_is[i]))
        L_state = ufl.rhs(pde_varf(u_trial, m, u_test, u_is[i]))

        state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
        dl.solve(state_A, u_list[i].vector(), state_b)

    # Evaluate new cost
    cost_new, misfit_new, reg_new = cost_multisource(u_list, d_list, m, beta)

    # Check Armijo condition
    if cost_new < cost_old - alpha * c_armijo * grad_norm2:
        cost_old = cost_new
        backtrack_converged = True
        break
    else:
        alpha *= 0.5
        m.assign(m_prev) # Reset m

if not backtrack_converged:
    print("Backtracking failed. A sufficient descent direction was not found")
    break

# Print progress
if (iter % print_any) == 0:
    print(f"{iter:3d} {cost_new:8.5e} {misfit_new:8.5e} {reg_new:8.5e} "
          f"{gradnorm:8.5e} {alpha:8.5e} {it_backtrack:3d}")

# Plot current solution
if (iter % plot_any) == 0:
    plt.figure(figsize=(15,5))
    nb.plot(m, subplot_loc=121, mytitle="Current m")
    nb.plot(u_list[0], subplot_loc=122, mytitle="u1(m)")
    plt.figure(figsize=(15,5))
    nb.plot(u_list[1], subplot_loc=121, mytitle="u2(m)")
    nb.plot(u_list[2], subplot_loc=122, mytitle="u3(m)")
    plt.show()

```

```

# Check convergence
if gradnorm < tol * gradnorm0 and iter > 0:
    converged = True
    print(f"Steepest descent converged in {iter} iterations")

# Prepare for next iteration
alpha *= 1.5 # Increase step size for next iteration
iter += 1

if not converged:
    print(f"Steepest descent did not converge in {maxiter} iterations")

# =====
# Results and Visualization
# =====
# Compute difference between true and recovered parameter
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector() - m.vector()

# Plot results
plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
plt.show()

# Compute L2 error
l2_norm_squared = dl.assemble((m_diff**2)*dl.dx)
print("Multiple sources, L2-inner product, k0 = 10, ||m - m true||^2 =", l2_norm_squared)

```

Repeated for $k_0 = 5$.

III.1

Once optimal regularization has been identified:

```

import matplotlib.pyplot as plt
%matplotlib inline

import hippylib as hp
import dolfin as dl
import ufl

from hippylib import nb

import numpy as np
import logging

logging.getLogger('FFC').setLevel(logging.WARNING)
logging.getLogger('UFL').setLevel(logging.WARNING)
dl.set_log_active(False)

np.random.seed(seed=42)

```

Class HessianOperator to perform Hessian apply to a vector

```

class HessianOperator():
    cgiter = 0
    def __init__(self, R, Wmm, C, A, adj_A, W, Wum, use_gaussnewton=False):
        self.R = R
        self.Wmm = Wmm
        self.C = C
        self.A = A
        self.adj_A = adj_A
        self.W = W
        self.Wum = Wum
        self.use_gaussnewton = use_gaussnewton

        # incremental state
        self.du = dl.Vector()
        self.A.init_vector([self.du, 0])

        # incremental adjoint
        self.dp = dl.Vector()
        self.adj_A.init_vector(self.dp, 0)

        # auxiliary vector
        self.Wum_du = dl.Vector()
        self.Wum.init_vector(self.Wum_du, 1)

    def init_vector(self, v, dim):
        self.R.init_vector(v, dim)

    # Hessian performed on v, output as generic vector y
    def mult(self, v, y):
        self.cgiter += 1
        y.zero()
        if self.use_gaussnewton:
            self.mult_GaussNewton(v, y)
        else:
            self.mult_Newton(v, y)

    # define (Gauss-Newton) Hessian apply  $H * v$ 
    def mult_GaussNewton(self, v, y):
        # incremental forward
        rhs = -(self.C * v)
        dl.solve (self.A, self.du, rhs)

        # incremental adjoint
        rhs = - (self.W * self.du)
        dl.solve (self.adj_A, self.dp, rhs)

        # Misfit term
        self.C.transpmult(self.dp, y)

        if self.R:
            Rv = self.R*v
            y.axpy(1, Rv)

    # define (Newton) Hessian apply  $H * v$ 
    def mult_Newton(self, v, y):
        # incremental forward
        rhs = -(self.C * v)
        dl.solve (self.A, self.du, rhs)

        # incremental adjoint
        rhs = -(self.W * self.du) - self.Wum * v
        dl.solve (self.adj_A, self.dp, rhs)

        # Misfit term
        self.C.transpmult(self.dp, y)

        self.Wum.transpmult(self.du, self.Wum_du)
        y.axpy(1., self.Wum_du)

        y.axpy(1., self.Wmm*v)

        # Reg/Prior term
        if self.R:
            y.axpy(1., self.R*v)

```



```

# Results storage
results = []

for nrefinements in range(3):
    print(f"\n\n== Solving for refinement level {nrefinements} ==")

    # Create mesh and define function spaces
    mesh = d1.Mesh("circle.xml")

    for _ in range(nrefinements):
        mesh = d1.refine(mesh)

    print(f"Refinement level {nrefinements}; Number of mesh elements: {mesh.num_cells()}")

    Vm = d1.FunctionSpace(mesh, 'Lagrange', 1)
    Vu = d1.FunctionSpace(mesh, 'Lagrange', 2)

    # Initialize input functions
    k0 = 10.
    k0squared = d1.Constant(k0*k0)

    nsources = 3
    theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
    radius = 1.05
    s0 = radius*np.cos(theta_view)
    s1 = radius*np.sin(theta_view)

    u_is = []
    for i in np.arange(nsources):
        distance_source = d1.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1))",
                                         s0=s0[i], s1=s1[i], degree=1)
        u_i_exp = d1.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                                k0=k0, distance_source=distance_source, degree=6)
        u_is.append(d1.interpolate(u_i_exp, Vu))

    encoding = np.random.normal(size=nsources)
    u_inc = d1.Function(Vu)
    for i, u_i in enumerate(u_is):
        u_inc.vector().axpy(encoding[i], u_i.vector())

    # PDE variational form
    def pde_varf(u, m, p):
        return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
                - k0squared*(d1.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
                - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

    # Create synthetic true parameter and observations
    # The true parameter
    mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
    mtrue = d1.interpolate(d1.Expression(mtrue_str, degree=5), Vm)

    bc_state = []

    # Noise Level
    noise_level = 0.01

    u_trial = d1.TrialFunction(Vu)
    u_test = d1.TestFunction(Vu)

    # Solve forward problem for synthetic data
    a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test))
    L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test))
    A_true, b_true = d1.assemble_system(a_true, L_true, bc_state)
    utrue = d1.Function(Vu)
    d1.solve(A_true, utrue.vector(), b_true)

    d = d1.Function(Vu)
    d.assign(utrue)
    MAX = d.vector().norm("linf")
    noise = d1.Vector()
    A_true.init_vector(noise, 1)
    noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local()))
    d.vector().axpy(1., noise)

    # Define functions for state, parameter, adjoint
    u = d1.Function(Vu)
    m = d1.Function(Vm)
    p = d1.Function(Vu)

    # Define Trial and Test Functions
    m_trial, p_trial = d1.TrialFunction(Vm), d1.TrialFunction(Vu)
    m_test, p_test = d1.TestFunction(Vm), d1.TestFunction(Vu)

    # Regularization parameter
    beta = 1e-7

    # Cost function
    def cost(u, d, m, beta):
        reg = 0.5*beta * d1.assemble(ufl.inner(ufl.grad(m), ufl.grad(m))*ufl.dx)
        misfit = 0.5 * d1.assemble((u-d)**2*ufl.dx)
        return [reg + misfit, misfit, reg]

```

```

# Weak forms
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx

a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx

grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# Initial guess
m0 = dl.Constant(0)
m.assign(m0)

# Solve state equation
state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
dl.solve(state_A, u.vector(), state_b)

# Evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# Define other variational forms
W_varf = ufl.inner(u_trial, u_test) * ufl.dx
R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

# Assemble constant matrices
W = dl.assemble(W_varf)
R = dl.assemble(R_varf)

# Optimization parameters
tol = 1e-8
c = 1e-4
maxiter = 30
plot_on = False

# Initialize counters
iter = 1
total_cg_iter = 0
gauss_newton_iters = 0
converged = False
g, m_delta = dl.Vector(), dl.Vector()
R.init_vector(m_delta, 0)
R.init_vector(g, 0)
m_prev = dl.Function(Vm)

print("Nit   CGit   cost           misfit       reg           sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
    tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
    Psolver.set_operator(P)

```

```

solver = hp.CGsolverSteihaug()
solver.set_operator(Hess_Apply)
solver.set_preconditioner(Psolver)
solver.parameters["rel_tolerance"] = tol_cg
solver.parameters["zero_initial_guess"] = True
solver.parameters["print_level"] = -1

# Solve Newton system
solver.solve(m_delta, -MG)
total_cg_iter += Hess_Apply.cgiter

# Line search
alpha = 1
descent = 0
no_backtrack = 0
m_prev.assign(m)
while descent == 0 and no_backtrack < 10:
    m.vector().axpy(alpha, m_delta)
    state_A, state_b = d1.assemble_system(a_state, L_state)
    d1.solve(state_A, u.vector(), state_b)
    [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

    if cost_new < cost_old + alpha * c * MG.inner(m_delta):
        cost_old = cost_new
        descent = 1
    else:
        no_backtrack += 1
        alpha *= 0.5
        m.assign(m_prev)

graddir = np.sqrt(-MG.inner(m_delta))

print(f"{iter:2d} {' '}{Hess_Apply.cgiter:2d} {' '}{cost_new:8.5e} {' '}\
      {misfit_new:8.5e} {' '}{reg_new:8.5e} {' '}{graddir:8.5e} {' '}{gradnorm:8.5e} {' '}{alpha:5.2f} {' '}{tolcg:5.3e}")

if plot_on:
    nb.multitl_plot([m,u,p], ["m","u","p"], same_colorbar=False)
    plt.show()

# Check convergence
if gradnorm < tol and iter > 1:
    converged = True
    print(f"Newton's method converged in {iter} iterations")
    print(f"Total number of CG iterations: {total_cg_iter}")
    print(f"Gauss-Newton iterations: {gauss_newton_iters}")

iter += 1

```

```

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Plot results for this refinement level
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]

plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
print(f"Refinement level {nrefinements}")
plt.show()

# Store results
results.append({
    'refinement': nrefinements,
    'total_iterations': iter-1,
    'gauss_newton_iterations': gauss_newton_iters,
    'total_cg_iterations': total_cg_iter,
    'final_cost': cost_new,
    'final_gradnorm': gradnorm,
    'm_diff_norm': dl.norm(m_diff, 'L2')
})

# Print summary of results
print("\n\n=== Summary of Results ===")
print("Refinement | Total Iters | GN Iters | Total CG Iters | Final Cost | Final GradNorm | m_diff L2")
for res in results:
    # print(f"{res['refinement']:10d} | {res['total_iterations']:11d} | {res['gauss_newton_iterations']:8d} | "
    print(
        f"{res['refinement']:10d} | {res['total_iterations']:11d} | {res['gauss_newton_iterations']:8d} | "
        f"{res['total_cg_iterations']:13d} | {res['final_cost']:10.2e} | {res['final_gradnorm']:12.2e} | "
        f"{res['m_diff_norm']:8.2e}"
    )

```

To identify the optimal regularization:

```

# Results storage
results = []

# Range of beta values to test
beta_values = np.logspace(-8, -2, 7) # From 1e-6 to 1e-2

for nrefinements in range(3):
    print(f"\n\n=== Solving for refinement level {nrefinements} ===")

    # Create mesh and define function spaces
    mesh = dl.Mesh("circle.xml")
    for _ in range(nrefinements):
        mesh = dl.refine(mesh)

    print(f"Refinement level {nrefinements}; Number of mesh elements: {mesh.num_cells()}")
    Vm = dl.FunctionSpace(mesh, 'Lagrange', 1)
    Vu = dl.FunctionSpace(mesh, 'Lagrange', 2)

    # Initialize input functions
    k0 = 10.
    k0squared = dl.Constant(k0*k0)

    nsources = 3
    theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
    radius = 1.05
    s0 = radius*np.cos(theta_view)
    s1 = radius*np.sin(theta_view)

    u_is = []
    for i in np.arange(nsources):
        distance_source = dl.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1) )",
                                         s0=s0[i], s1=s1[i], degree=1)
        u_i_exp = dl.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                                k0=k0, distance_source=distance_source, degree=6)
        u_is.append(dl.interpolate(u_i_exp, Vu))

    encoding = np.random.normal(size=nsources)
    u_inc = dl.Function(Vu)
    for i, u_i in enumerate(u_is):
        u_inc.vector().axpy(encoding[i], u_i.vector())

```



```

# PDE variational form
def pde_varf(u, m, p):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(dl.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# Create synthetic true parameter and observations
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

bc_state = []
noise_level = 0.01

u_trial = dl.TrialFunction(Vu)
u_test = dl.TestFunction(Vu)

# Solve forward problem for synthetic data
a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test))
L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test))
A_true, b_true = dl.assemble_system(a_true, L_true, bc_state)
utru = dl.Function(Vu)
dl.solve(A_true, utru.vector(), b_true)

d = dl.Function(Vu)
d.assign(utru)
MAX = d.vector().norm("linf")
noise = dl.Vector()
A_true.init_vector(noise, 1)
noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local())))
d.vector().axpy(1., noise)

# Define functions for state, parameter, adjoint
u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# Define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)

```

```

# Cost function
def cost(u, d, m, beta):
    reg = 0.5*beta * dl.assemble(ufl.inner(ufl.grad(m), ufl.grad(m))*ufl.dx)
    misfit = 0.5 * dl.assemble((u-d)**2*ufl.dx)
    return [reg + misfit, misfit, reg]

# Weak forms (these don't depend on beta)
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# First, find the optimal beta by testing multiple values
beta_test_results = []
for beta in beta_values:
    print(f"\nTesting beta = {beta:.2e}")

    # Initialize with zero
    m0 = dl.Constant(0)
    m.assign(m0)

    # Reassemble beta-dependent forms
    grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
    R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
    R = dl.assemble(R_varf)

    # Solve state equation
    state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
    dl.solve(state_A, u.vector(), state_b)

    # Evaluate cost
    [cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

    # Define other variational forms
    W_varf = ufl.inner(u_trial, u_test) * ufl.dx
    C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
    Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
    Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

W = dl.assemble(W_varf)

# Optimization parameters
tol = 1e-8
c = 1e-4
maxiter = 20
plot_on = False

# Initialize counters
iter = 1
total_cg_iter = 0
gauss_newton_iters = 0
converged = False
g, m_delta = dl.Vector(), dl.Vector()
R.init_vector(m_delta, 0)
R.init_vector(g, 0)
m_prev = dl.Function(Vm)

print("Nit    CGit    cost          misfit          reg          sqrt(-G*D)    ||grad||          alpha    tolcg")

```

```

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
    tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
    Psolver.set_operator(P)

    solver = hp.CGSolverSteihaug()
    solver.set_operator(Hess_Apply)
    solver.set_preconditioner(Psolver)
    solver.parameters["rel_tolerance"] = tolcg
    solver.parameters["zero_initial_guess"] = True
    solver.parameters["print_level"] = -1

    # Solve Newton system
    solver.solve(m_delta, -MG)
    total_cg_iter += Hess_Apply.cgiter

    # Line search
    alpha = 1
    descent = 0
    no_backtrack = 0
    m_prev.assign(m)
    while descent == 0 and no_backtrack < 10:
        m.vector().axpy(alpha, m_delta)
        state_A, state_b = dl.assemble_system(a_state, L_state)
        dl.solve(state_A, u.vector(), state_b)
        [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

        if cost_new < cost_old + alpha * c * MG.inner(m_delta):
            cost_old = cost_new
            descent = 1
        else:
            no_backtrack += 1
            alpha *= 0.5
            m.assign(m_prev)

    graddir = np.sqrt(-MG.inner(m_delta))

    print(f"iter:2d} {' '} {Hess_Apply.cgiter:2d} {' '} {cost_new:8.5e} {' '}
        {misfit_new:8.5e} {' '} {reg_new:8.5e} {' '} {graddir:8.5e} {' '} {gradnorm:8.5e} {' '} {alpha:5.2f} {' '} {tolcg:5.3e}")

    if plot_on:
        nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
        plt.show()

    # Check convergence
    if gradnorm < tol and iter > 1:
        converged = True
        print(f"Newton's method converged in {iter} iterations")
        print(f"Total number of CG iterations: {total_cg_iter}")
        print(f"Gauss-Newton iterations: {gauss_newton_iters}")

    iter += 1

```

```

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Compute difference with true solution
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]
diff_norm = dl.norm(m_diff, 'L2')

beta_test_results.append([
    'beta': beta,
    'm_diff_norm': diff_norm,
    'm': dl.Function(Vm, m.vector().copy()),
    'iterations': iter
])

print(f"Beta {beta:.2e} -> Error norm: {diff_norm:.4e}")

plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
plt.show()

# Find the beta with minimum difference to true solution
optimal_idx = np.argmin([res['m_diff_norm'] for res in beta_test_results])
optimal_beta = beta_test_results[optimal_idx]['beta']
optimal_m = beta_test_results[optimal_idx]['m']

print(f"\nOptimal beta found: {optimal_beta:.2e} with error norm {beta_test_results[optimal_idx]['m_diff_norm']:.4e}")

# Now run full optimization with optimal beta to get detailed statistics
print("\nRunning full optimization with optimal beta...")
beta = optimal_beta

# Reinitialize
m0 = dl.Constant(0)
m.assign(m0)

```

```

# Reassemble beta-dependent forms
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Solve state equation
state_A, state_b = dl.assemble_system(a_state, l_state, bc_state)
dl.solve(state_A, u.vector(), state_b)

# Evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# Define other variational forms
W_varf = ufl.inner(u_trial, u_test) * ufl.dx
C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

W = dl.assemble(W_varf)

# Optimization parameters
tol = 1e-8
c = 1e-4
maxiter = 40
plot_on = False

# Initialize counters
iter = 1
total_cg_iter = 0
gauss_newton_iters = 0
converged = False
g, m_delta = dl.Vector(), dl.Vector()
R.init_vector(m_delta, 0)
R.init_vector(g, 0)
m_prev = dl.Function(Vm)

print("Nit   CGit   cost       misfit       reg       sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, l_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
        tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
    Psolver.set_operator(P)

    solver = hp.CGSolverSteihaug()
    solver.set_operator(Hess_Apply)
    solver.set_preconditioner(Psolver)
    solver.parameters["rel_tolerance"] = tolcg
    solver.parameters["zero_initial_guess"] = True
    solver.parameters["print_level"] = -1

```



```

# Solve Newton system
solver.solve(m_delta, -MG)
total_cg_iter += Hess_Apply.cgiter

# Line search
alpha = 1
descent = 0
no_backtrack = 0
m_prev.assign(m)
while descent == 0 and no_backtrack < 10:
    m.vector().axpy(alpha, m_delta)
    state_A, state_b = dl.assemble_system(a_state, l_state)
    dl.solve(state_A, u.vector(), state_b)
    [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

    if cost_new < cost_old + alpha * c * MG.inner(m_delta):
        cost_old = cost_new
        descent = 1
    else:
        no_backtrack += 1
        alpha *= 0.5
        m.assign(m_prev)

graddir = np.sqrt(-MG.inner(m_delta))

print(f"{{iter:2d}} {{' '}} {{Hess_Apply.cgiter:2d}} {{' '}} {{cost_new:8.5e}} {{' '}}
      {{misfit_new:8.5e}} {{' '}} {{reg_new:8.5e}} {{' '}} {{graddir:8.5e}} {{' '}} {{gradnorm:8.5e}} {{' '}} {{alpha:5.2f}} {{' '}} {{tolcg:5.3e}}")

if plot_on:
    nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
    plt.show()

# Check convergence
if gradnorm < tol and iter > 1:
    converged = True
    print(f"Newton's method converged in {{iter}} iterations")
    print(f"Total number of CG iterations: {{total_cg_iter}}")
    print(f"Gauss-Newton iterations: {{gauss_newton_iters}}")

iter += 1

```

```

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Compute final difference with true solution
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]

# Plot results
plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", f"Recovered m ( $\beta$ ={optimal_beta:.1e})", "Difference"],
    same_colorbar=False
)
plt.suptitle(f"Refinement level {nrefinements}")
plt.show()

# Plot beta vs error norm
plt.figure(figsize=(8, 5))
plt.semilogx(beta_values, [res['m_diff_norm'] for res in beta_test_results], 'b.-')
plt.semilogx(optimal_beta, beta_test_results[optimal_idx]['m_diff_norm'], 'ro', label='Optimal beta')
plt.xlabel('Regularization parameter  $\beta$ ')
plt.ylabel('Error norm ( $m_{true} - m$ )')
plt.title(f'Error vs Regularization (Refinement {nrefinements})')
plt.grid(True, which="both", ls="--")
plt.legend()
plt.show()

# Store results
results.append({
    'refinement': nrefinements,
    'optimal_beta': optimal_beta,
    'total_iterations': iter-1,
    'gauss_newton_iterations': gauss_newton_iters,
    'total_cg_iterations': total_cg_iter,
    'final_cost': cost_new,
    'final_gradnorm': gradnorm,
    'm_diff_norm': dl.norm(m_diff, 'L2'),
    'mesh_size': mesh.num_cells()
})

# Print summary of results
print("\n\n=== Summary of Results ===")
print("Refinement | Mesh Size | Optimal  $\beta$  | Total Iters | GN Iters | Total CG Iters | Final Cost | Final GradNorm | m_diff L2")
for res in results:
    print(f"{res['refinement']:10d} | {res['mesh_size']:9d} | {res['optimal_beta']:8.2e} | {res['total_iterations']:11d} | "
          f"{res['gauss_newton_iterations']:8d} | {res['total_cg_iterations']:13d} | {res['final_cost']:10.2e} | "
          f"{res['final_gradnorm']:12.2e} | {res['m_diff_norm']:8.2e}")

```

III.2.a

Same imports and class HessianOperator.

```
from hippylib import MultiVector, Random, doublePassd

# Fixed parameters
beta_values = [1e-4]
k0 = 10.
noise_level = 0.01
nsources = 3

# Results storage for eigenvalues
eigen_results_a = []

for nrefinements in range(3):
    print(f"\n=== Solving for refinement level {nrefinements} ===")

    # Create mesh and define function spaces
    mesh = dl.Mesh("circle.xml")
    for _ in range(nrefinements):
        mesh = dl.refine(mesh)

    print(f"Refinement level {nrefinements}; Number of mesh elements: {mesh.num_cells()}")
    Vm = dl.FunctionSpace(mesh, 'Lagrange', 1)
    Vu = dl.FunctionSpace(mesh, 'Lagrange', 2)

    # Initialize input functions
    k0squared = dl.Constant(k0*k0)

    # Create sources
    theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
    radius = 1.05
    s0 = radius*np.cos(theta_view)
    s1 = radius*np.sin(theta_view)

    u is = []
```

```

for i in np.arange(nsources):
    distance_source = dl.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1) )",
                                     s0=s0[i], s1=s1[i], degree=1)
    u_i_exp = dl.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                             k0=k0, distance_source=distance_source, degree=6)
    u_is.append(dl.interpolate(u_i_exp, Vu))

encoding = np.random.normal(size=nsources)
u_inc = dl.Function(Vu)
for i, u_i in enumerate(u_is):
    u_inc.vector().axpy(encoding[i], u_i.vector())

# PDE variational form
def pde_varf(u, m, p):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(dl.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# Create synthetic true parameter and observations
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

bc_state = []
noise_level = 0.01

u_trial = dl.TrialFunction(Vu)
u_test = dl.TestFunction(Vu)

# Solve forward problem for synthetic data
a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test))
L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test))
A_true, b_true = dl.assemble_system(a_true, L_true, bc_state)
utru = dl.Function(Vu)
dl.solve(A_true, utru.vector(), b_true)

d = dl.Function(Vu)
d.assign(utru)
MAX = d.vector().norm("linf")
noise = dl.Vector()
A_true.init_vector(noise, 1)
noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local()))))
d.vector().axpy(1., noise)

```

```

# Define functions for state, parameter, adjoint
u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# Define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)

# Cost function
def cost(u, d, m, beta):
    reg = 0.5*beta * dl.assemble(ufl.inner(ufl.grad(m), ufl.grad(m))*ufl.dx)
    misfit = 0.5 * dl.assemble((u-d)**2*ufl.dx)
    return [reg + misfit, misfit, reg]

# Weak forms
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# First, find the optimal beta by testing multiple values
beta_test_results = []
for beta in beta_values:
    print(f"\nTesting beta = {beta:.2e}")

    # Initialize with zero
    m0 = dl.Constant(0)
    m.assign(m0)

    # Reassemble beta-dependent forms
    grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
    R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
    R = dl.assemble(R_varf)

    # Solve state equation
    state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
    dl.solve(state_A, u.vector(), state_b)

    # Evaluate cost
    [cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

    # Define other variational forms
    W_varf = ufl.inner(u_trial, u_test) * ufl.dx
    C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
    Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
    Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

    W = dl.assemble(W_varf)

    # Optimization parameters
    tol = 1e-8
    c = 1e-4
    maxiter = 20
    plot_on = False

    # Initialize counters
    iter = 1
    total_cg_iter = 0
    gauss_newton_iters = 0
    converged = False
    g, m_delta = dl.Vector(), dl.Vector()
    R.init_vector(m_delta, 0)
    R.init_vector(g, 0)
    m_prev = dl.Function(Vm)

```



```

print("Nit   CGit   cost           misfit       reg           sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
    tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", "amg")
    Psolver.set_operator(P)

    solver = hp.CGSolverSteihaug()
    solver.set_operator(Hess_Apply)
    solver.set_preconditioner(Psolver)
    solver.parameters["rel_tolerance"] = tolcg
    solver.parameters["zero_initial_guess"] = True
    solver.parameters["print_level"] = -1

    # Solve Newton system
    solver.solve(m_delta, -MG)
    total_cg_iter += Hess_Apply.cgiter

```

```

# Line search
alpha = 1
descent = 0
no_backtrack = 0
m_prev.assign(m)
while descent == 0 and no_backtrack < 10:
    m.vector().axpy(alpha, m_delta)
    state_A, state_b = dl.assemble_system(a_state, l_state)
    dl.solve(state_A, u.vector(), state_b)
    [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

    if cost_new < cost_old + alpha * c * MG.inner(m_delta):
        cost_old = cost_new
        descent = 1
    else:
        no_backtrack += 1
        alpha *= 0.5
        m.assign(m_prev)

graddir = np.sqrt(-MG.inner(m_delta))

print(f"iter:2d { ' ' } {Hess_Apply.cgiter:2d} { ' ' } {cost_new:8.5e} { ' ' }
      {misfit_new:8.5e} { ' ' } {reg_new:8.5e} { ' ' } {graddir:8.5e} { ' ' } {gradnorm:8.5e} { ' ' } {alpha:5.2f} { ' ' } {tolcg:5.3e}")

if plot_on:
    nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
    plt.show()

# Check convergence
if gradnorm < tol and iter > 1:
    converged = True
    print(f"Newton's method converged in {iter} iterations")
    print(f"Total number of CG iterations: {total_cg_iter}")
    print(f"Gauss-Newton iterations: {gauss_newton_iters}")

    iter += 1

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Compute difference with true solution
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]
diff_norm = dl.norm(m_diff, 'L2')

```

```

beta_test_results.append({
    'beta': beta,
    'm_diff_norm': diff_norm,
    'm': dl.Function(Vm, m.vector().copy()),
    'iterations': iter
})

print(f"Beta {beta:.2e} -> Error norm: {diff_norm:.4e}")

plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
plt.show()

# At the solution, compute eigenvalues using double pass algorithm
m_opt = dl.Function(Vm)
m_opt.assign(m)

# Assemble components for Hessian
R_varf = beta * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Mass matrix
M_varf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(M_varf)

# Preconditioner
P = R + 0.1*beta * M
Psolver = dl.PETScKrylovSolver("cg", "amg")
Psolver.set_operator(P)

# Create misfit Hessian operator (Gauss-Newton approximation)
Hmisfit = HessianOperator(None, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=False)

# Parameters for double pass algorithm
k = 1000 # Number of eigenvalues to compute
p = 20 # Oversampling parameter

```

```

# Create random vectors for double pass
Omega = MultiVector(m_opt.vector(), k+p)
parRandom = Random()
parRandom.normal(1., Omega)

# Compute eigenvalues using double pass algorithm
lmbda, evecs = doublePassG(Hmisfit, P, Psolver, Omega, k)

# Store eigenvalues
eigen_results_a.append({
    'refinement': nrefinements,
    'mesh_size': mesh.num_cells(),
    'eigenvalues': lmbda,
    'eigenvectors': evecs
})

# Plot eigenvalues
plt.figure()
plt.plot(range(0,k), lmbda, 'b*', range(0,k+1), np.ones(k+1), '-r')
plt.yscale('log')
plt.xlabel('Eigenvalue index')
plt.ylabel('Magnitude')
plt.title(f'Eigenvalues at refinement {nrefinements}')
plt.grid(True)
plt.show()

# Plot some eigenvectors
nb.plot_eigenvectors(Vm, evecs, mytitle="Eigenvectors", which=[0,1,2,5,10,15])
plt.show()

# Print summary
print("\n=== Part (a) Results ===")
for res in eigen_results_a:
    print(f"\nRefinement {res['refinement']}, Mesh size {res['mesh_size']}")
    print(f"First 5 eigenvalues: {res['eigenvalues'][:5]}")
    print(f>Last 5 eigenvalues: {res['eigenvalues'][-5:]}")
    print(f"Condition number: {max(res['eigenvalues'])/min(res['eigenvalues']):.2e}")

```

For regularized-preconditioned hessian eigenvalues:

```
Hmisfit = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum,
use_gaussnewton=False)
```

III.2.b

For regularized-preconditioned hessian eigenvalues, use the below:

```
Hmisfit = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum,
use_gaussnewton=False)
```

```

# Fixed parameters for part (b)
from hippylib import MultiVector, Random, doublePassG
nrefinements = 0 # Coarse mesh only
beta = 1e-4
k0_values = [5., 10., 12.5]
noise_level = 0.01
nsources = 3

# Results storage for eigenvalues
eigen_results_b = []

# Create mesh (coarse only)
mesh = dl.Mesh("circle.xml")
Vm = dl.FunctionSpace(mesh, 'Lagrange', 1)
Vu = dl.FunctionSpace(mesh, 'Lagrange', 2)

m_prev = None # For warm-starting

for k0 in k0_values:
    print(f"\n=== Solving for k0 = {k0} ===")
    k0squared = dl.Constant(k0*k0)

    # Create sources
    theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
    radius = 1.05
    s0 = radius*np.cos(theta_view)
    s1 = radius*np.sin(theta_view)

    u_is = []
    for i in np.arange(nsources):
        distance_source = dl.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1) )",
                                         s0=s0[i], s1=s1[i], degree=1)
        u_i_exp = dl.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                                k0=k0, distance_source=distance_source, degree=6)
        u_is.append(dl.interpolate(u_i_exp, Vu))

    encoding = np.random.normal(size=nsources)
    u_inc = dl.Function(Vu)
    for i, u_i in enumerate(u_is):
        u_inc.vector().axpy(encoding[i], u_i.vector())

```

```

# PDE variational form
def pde_varf(u, m, p):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(dl.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# Create synthetic true parameter and observations
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

bc_state = []
u_trial = dl.TrialFunction(Vu)
u_test = dl.TestFunction(Vu)

# Solve forward problem for synthetic data
a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test))
L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test))
A_true, b_true = dl.assemble_system(a_true, L_true, bc_state)
utru = dl.Function(Vu)
dl.solve(A_true, utru.vector(), b_true)

# Add noise to data
d = dl.Function(Vu)
d.assign(utru)
MAX = d.vector().norm("linf")
noise = dl.Vector()
A_true.init_vector(noise, 1)
noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local()))))
d.vector().axpy(1., noise)

# Define functions for state, parameter, adjoint
u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# Define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)

# Cost function
def cost(u, d, m, beta):
    reg = 0.5*beta * dl.assemble(ufl.inner(ufl.grad(m), ufl.grad(m))*ufl.dx)
    misfit = 0.5 * dl.assemble((u-d)**2*ufl.dx)
    return [reg + misfit, misfit, reg]

```



```

# Weak forms
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx
grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# Initialize with zero or previous solution
if m_prev is None:
    m0 = dl.Constant(0)
    m.assign(m0)
else:
    print("bef")
    m.assign(m_prev)
    print("ass")

# Reassemble beta-dependent forms
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Optimization parameters
tol = 1e-8
c = 1e-4
maxiter = 20
plot_on = False

# Initialize counters
iter = 1
total_cg_iter = 0
gauss_newton_iters = 0
converged = False
g, m_delta = dl.Vector(), dl.Vector()
R.init_vector(m_delta, 0)
R.init_vector(g, 0)
m_prev = dl.Function(Vm)

```

```

print("Nit   CGit   cost       misfit       reg       sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve state equation
    state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
    dl.solve(state_A, u.vector(), state_b)

    # Evaluate cost
    [cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
    tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    W_varf = ufl.inner(u_trial, u_test) * ufl.dx
    C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
    Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
    Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

    W = dl.assemble(W_varf)
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

```

```

# Define Hessian apply operator
Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
P = R + 0.1*beta * M
Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
Psolver.set_operator(P)

solver = hp.CGSolverSteihaug()
solver.set_operator(Hess_Apply)
solver.set_preconditioner(Psolver)
solver.parameters["rel_tolerance"] = tolcg
solver.parameters["zero_initial_guess"] = True
solver.parameters["print_level"] = -1

# Solve Newton system
solver.solve(m_delta, -MG)
total_cg_iter += Hess_Apply.cgiter

# Line search
alpha = 1
descent = 0
no_backtrack = 0
m_prev.assign(m)
while descent == 0 and no_backtrack < 10:
    m.vector().axpy(alpha, m_delta)
    state_A, state_b = dl.assemble_system(a_state, l_state)
    dl.solve(state_A, u.vector(), state_b)
    [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

    if cost_new < cost_old + alpha * c * MG.inner(m_delta):
        cost_old = cost_new
        descent = 1
    else:
        no_backtrack += 1
        alpha *= 0.5
        m.assign(m_prev)

graddir = np.sqrt(-MG.inner(m_delta))

print(f"iter:2d { ' ' } {Hess_Apply.cgiter:2d} { ' ' } {cost_new:8.5e} { ' ' } {misfit_new:8.5e} { ' ' } {reg_new:8.5e} { ' ' } {graddir:8.5e} { ' ' } {gradnorm:8.5e} { ' ' } {alpha:5.2f} { ' ' } {tolcg:5.3e}")

# Check convergence
if gradnorm < tol and iter > 1:
    converged = True
    print(f"Newton's method converged in {iter} iterations")
    print(f"Total number of CG iterations: {total_cg_iter}")
    print(f"Gauss-Newton iterations: {gauss_newton_iters}")

    iter += 1

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# At the solution, compute eigenvalues using double pass algorithm
m_opt = dl.Function(Vm)
m_opt.assign(m)

# Assemble components for Hessian
R_varf = beta * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Mass matrix
M_varf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(M_varf)

# Preconditioner
P = R + 0.1*beta * M
Psolver = dl.PETScKrylovSolver("cg", "amg")
Psolver.set_operator(P)

# Create misfit Hessian operator (Gauss-Newton approximation)
Hmisfit = HessianOperator(None, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=False)

# Parameters for double pass algorithm
k = 100 # Number of eigenvalues to compute
p = 20 # Oversampling parameter

```

```

# Create random vectors for double pass
Omega = MultiVector(m_opt.vector(), k+p)
parRandom = Random()
parRandom.normal(1., Omega)

# Compute eigenvalues using double pass algorithm
lmbda, evecs = doublePassG(Hmisfit, P, Psolver, Omega, k)

# # Store eigenvalues
# eigen_results_a.append({
#     'k0': k0,
#     'refinement': nrefinements,
#     'mesh_size': mesh.num_cells(),
#     'eigenvalues': lmbda,
#     'eigenvectors': evecs
# })

# Plot eigenvalues
# print(evecs)
plt.figure()
plt.plot(range(0,k), lmbda, 'b*', range(0,k+1), np.ones(k+1), '-r')
plt.yscale('log')
plt.xlabel('Eigenvalue index')
plt.ylabel('Magnitude')
plt.title(f'Eigenvalues at refinement {nrefinements}, k0 = {k0}')
plt.grid(True)
# print(2)

eigen_results_b.append({
    'k0': k0,
    'eigenvalues': lmbda
})

# Store solution for next warm start
m_prev = dl.Function(Vm)
m_prev.assign(m_opt)

# Plot eigenvalues
# plt.semilogy(evecs, 'o-')
print("done plot")

# Print summary
print("\n=== Part (b) Results ===")
for res in eigen_results_b:
    print(f"\nWave number k0 = {res['k0']}")
    print(f"First 5 eigenvalues: {res['eigenvalues'][:5]}")
    print(f>Last 5 eigenvalues: {res['eigenvalues'][-5:]}")
    print(f"Condition number: {max(res['eigenvalues'])/min(res['eigenvalues']):.2e}")

```

III.3

Same imports as above, along with the class HessianOperator.

```

# Results storage
results = []

# Range of beta values to test
beta_values = np.logspace(-6, -2, 7) # From 1e-6 to 1e-2
epsilon = 1e-2
for nrefinements in range(3):
    print(f"\n\n=== Solving for refinement level {nrefinements} ===")

    # Create mesh and define function spaces
    mesh = dl.Mesh("circle.xml")
    for _ in range(nrefinements):
        mesh = dl.refine(mesh)

    print(f"Refinement level {nrefinements}; Number of mesh elements: {mesh.num_cells()}")
    Vm = dl.FunctionSpace(mesh, 'Lagrange', 1)
    Vu = dl.FunctionSpace(mesh, 'Lagrange', 2)

    # Initialize input functions
    k0 = 10.
    k0squared = dl.Constant(k0*k0)

    nsources = 3
    theta_view = np.linspace(-np.pi, np.pi, nsources, endpoint=False)
    radius = 1.05
    s0 = radius*np.cos(theta_view)
    s1 = radius*np.sin(theta_view)

    u_is = []
    for i in np.arange(nsources):
        distance_source = dl.Expression("std::sqrt((x[0]-s0)*(x[0]-s0) + (x[1]-s1)*(x[1]-s1))",
                                         s0=s0[i], s1=s1[i], degree=1)
        u_i_exp = dl.Expression("-std::cos(k0*distance_source)/(4.*DOLFIN_PI*distance_source)",
                                k0=k0, distance_source=distance_source, degree=6)
        u_is.append(dl.interpolate(u_i_exp, Vu))

    encoding = np.random.normal(size=nsources)
    u_inc = dl.Function(Vu)
    for i, u_i in enumerate(u_is):
        u_inc.vector().axpy(encoding[i], u_i.vector())

# PDE variational form
def pde_varf(u, m, p):
    return (ufl.inner(ufl.grad(u), ufl.grad(p))*ufl.dx
            - k0squared*(dl.Constant(1.) - ufl.tanh(m))*u*p*ufl.dx
            - k0squared*ufl.tanh(m)*u_inc*p*ufl.dx)

# Create synthetic true parameter and observations
mtrue_str = '.2*((x[0]-.1)*(x[0]-.1) + 2.*(x[1]+.2)*(x[1]+.2) < 0.25)'
mtrue = dl.interpolate(dl.Expression(mtrue_str, degree=5), Vm)

bc_state = []
noise_level = 0.01

u_trial = dl.TrialFunction(Vu)
u_test = dl.TestFunction(Vu)

# Solve forward problem for synthetic data
a_true = ufl.lhs(pde_varf(u_trial, mtrue, u_test))
L_true = ufl.rhs(pde_varf(u_trial, mtrue, u_test))
A_true, b_true = dl.assemble_system(a_true, L_true, bc_state)
utru = dl.Function(Vu)
dl.solve(A_true, utru.vector(), b_true)

d = dl.Function(Vu)
d.assign(utru)
MAX = d.vector().norm("linf")
noise = dl.Vector()
A_true.init_vector(noise, 1)
noise.set_local(noise_level * MAX * np.random.normal(0, 1, len(d.vector().get_local()))))
d.vector().axpy(1., noise)

# Define functions for state, parameter, adjoint
u = dl.Function(Vu)
m = dl.Function(Vm)
p = dl.Function(Vu)

# Define Trial and Test Functions
m_trial, p_trial = dl.TrialFunction(Vm), dl.TrialFunction(Vu)
m_test, p_test = dl.TestFunction(Vm), dl.TestFunction(Vu)

```

```

def cost(u, d, m, beta):
    reg = beta * dl.assemble(ufl.sqrt(ufl.inner(ufl.grad(m), ufl.grad(m)) + epsilon)*ufl.dx) # TV regularization
    misfit = 0.5 * dl.assemble((u-d)**2*ufl.dx)
    return [reg + misfit, misfit, reg]
# Gradient of TV term - now correctly formulated
def grad_tv(m, m_test, beta, epsilon):
    grad_m = ufl.grad(m)
    norm_grad_m = ufl.sqrt(ufl.inner(grad_m, grad_m) + epsilon)
    return beta * ufl.inner(grad_m, ufl.grad(m_test)) / norm_grad_m * ufl.dx

# Weak forms (these don't depend on beta)
a_state = (ufl.inner(ufl.grad(u_trial), ufl.grad(u_test)) - (k0**2 * (1 - ufl.tanh(m)) * ufl.inner(u_trial, u_test))) * ufl.dx
L_state = (k0**2 * ufl.tanh(m) * ufl.inner(u_inc, u_test)) * ufl.dx
a_adj = (ufl.inner(ufl.grad(p_trial), ufl.grad(p_test)) - ufl.inner(p_trial, p_test)*k0**2*(1-ufl.tanh(m))) * ufl.dx
L_adj = - ufl.inner(u - d, p_test) * ufl.dx

# Mass matrix in parameter space
Mvarf = ufl.inner(m_trial, m_test) * ufl.dx
M = dl.assemble(Mvarf)

# First, find the optimal beta by testing multiple values
beta_test_results = []
for beta in beta_values:

    print(f"\nTesting beta = {beta:.2e}")

    # Initialize with zero
    m0 = dl.Constant(0)
    m.assign(m0)

    grad_misfit = (ufl.inner(m_test, p) * k0**2 * (1- ufl.tanh(m)**2) * (u - u_inc)) * ufl.dx
    grad_reg = grad_tv(m, m_test, beta, epsilon)
    grad_total = grad_misfit + grad_reg

    inner_terms1 = ufl.grad(ufl.grad(m_test)) * (ufl.inner(ufl.grad(m), ufl.grad(m)) + epsilon)**(-1/2)
    # inner_terms2 = -ufl.inner(ufl.inner(ufl.grad(m), (ufl.inner(ufl.grad(m), ufl.grad(m)) + epsilon)**(-3/2)), ufl.inner(ufl.grad(m),
    inner_terms2 = - (ufl.inner(ufl.grad(m), ufl.grad(m)) + epsilon)**(-3/2) * ufl.inner(ufl.grad(m), ufl.grad(m_test))

    # R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), inner_terms1+inner_terms2) * ufl.dx
    R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
    R = dl.assemble(R_varf)

    # Solve state equation
    state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
    dl.solve(state_A, u.vector(), state_b)

    # Evaluate cost
    [cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

    # Define other variational forms
    W_varf = ufl.inner(u_trial, u_test) * ufl.dx
    C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
    Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
    Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

    W = dl.assemble(W_varf)

    # Optimization parameters
    tol = 1e-8
    c = 1e-4
    maxiter = 50
    plot_on = False

    # Initialize counters
    iter = 1
    total_cg_iter = 0
    gauss_newton_iters = 0
    converged = False
    g, m_delta = dl.Vector(), dl.Vector()
    R.init_vector(m_delta, 0)
    R.init_vector(g, 0)
    m_prev = dl.Function(Vm)

```

```

print("Nit   CGit   cost           misfit       reg           sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
        tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
    Psolver.set_operator(P)

    solver = hp.CGSolverSteihaug()
    solver.set_operator(Hess_Apply)
    solver.set_preconditioner(Psolver)
    solver.parameters["rel_tolerance"] = tolcg
    solver.parameters["zero_initial_guess"] = True
    solver.parameters["print_level"] = -1

    # Solve Newton system
    solver.solve(m_delta, -MG)
    total_cg_iter += Hess_Apply.cgiter

    # Line search
    alpha = 1
    descent = 0
    no_backtrack = 0
    m_prev.assign(m)
    while descent == 0 and no_backtrack < 10:
        m.vector().axpy(alpha, m_delta)
        state_A, state_b = dl.assemble_system(a_state, L_state)
        dl.solve(state_A, u.vector(), state_b)
        [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

        if cost_new < cost_old + alpha * c * MG.inner(m_delta):
            cost_old = cost_new
            descent = 1
        else:
            no_backtrack += 1
            alpha *= 0.5
            m.assign(m_prev)

    graddir = np.sqrt(-MG.inner(m_delta))

    print(f"{{iter:2d}} {{' '}} {{Hess_Apply.cgiter:2d}} {{' '}} {{cost_new:8.5e}}
          {{' '}} {{misfit_new:8.5e}} {{' '}} {{reg_new:8.5e}} {{' '}} {{graddir:8.5e}} {{' '}} {{gradnorm:8.5e}} {{' '}} {{alpha:5.2f}} {{' '}} {{tolcg:5.3e}}")

    if plot_on:
        nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
        plt.show()

    # Check convergence
    if gradnorm < tol and iter > 1:
        converged = True
        print(f"Newton's method converged in {{iter}} iterations")
        print(f"Total number of CG iterations: {{total_cg_iter}}")
        print(f"Gauss-Newton iterations: {{gauss_newton_iters}}")

    iter += 1

```



```

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Compute difference with true solution
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]
diff_norm = dl.norm(m_diff, 'L2')

beta_test_results.append({
    'beta': beta,
    'm_diff_norm': diff_norm,
    'm': dl.Function(Vm, m.vector().copy()),
    'iterations': iter
})

print(f"Beta {beta:.2e} -> Error norm: {diff_norm:.4e}")

plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", "Recovered m", "Difference m - mtrue"],
    same_colorbar=False
)
plt.show()

# Find the beta with minimum difference to true solution
optimal_idx = np.argmin([res['m_diff_norm'] for res in beta_test_results])
optimal_beta = beta_test_results[optimal_idx]['beta']
optimal_m = beta_test_results[optimal_idx]['m']

print(f"\nOptimal beta found: {optimal_beta:.2e} with error norm {beta_test_results[optimal_idx]['m_diff_norm']:.4e}")

# Now run full optimization with optimal beta to get detailed statistics
print("\nRunning full optimization with optimal beta...")
beta = optimal_beta

# Reinitialize
m0 = dl.Constant(0)
m.assign(m0)

# Reassemble beta-dependent forms
grad_reg = dl.Constant(beta)*ufl.inner(ufl.grad(m), ufl.grad(m_test))*ufl.dx
# R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.inner(ufl.grad(m), ufl.grad(m)+epsilon)**(-1/2) * ufl.dx
# R_varf = dl.Constant(beta) * ufl.inner(ufl.grad(m_trial), ufl.grad(m_test)) * ufl.dx
R = dl.assemble(R_varf)

# Solve state equation
state_A, state_b = dl.assemble_system(a_state, L_state, bc_state)
dl.solve(state_A, u.vector(), state_b)

# Evaluate cost
[cost_old, misfit_old, reg_old] = cost(u, d, m, beta)

# Define other variational forms
W_varf = ufl.inner(u_trial, u_test) * ufl.dx
C_varf = ufl.inner(m_trial * k0**2 * (u - u_inc) * (1-(ufl.tanh(m)**2)), (u_test)) * ufl.dx
Wum_varf = m_trial * p * k0**2 * p_test * (1-ufl.tanh(m)**2) * ufl.dx
Wmm_varf = - (m_trial * p * k0**2 * (u-u_inc) * 2 * ufl.tanh(m) * (1- ufl.tanh(m)**2) * m_test) * ufl.dx

W = dl.assemble(W_varf)

# Optimization parameters
tol = 1e-8
c = 1e-4
maxiter = 50
plot_on = False

# Initialize counters
iter = 1
total_cg_iter = 0
gauss_newton_iters = 0
converged = False
g, m_delta = dl.Vector(), dl.Vector()
R.init_vector(m_delta, 0)
R.init_vector(g, 0)
m_prev = dl.Function(Vm)

```

```

print("Nit   CGit   cost           misfit       reg           sqrt(-G*D)   ||grad||       alpha   tolcg")

while iter < maxiter and not converged:
    # Solve adjoint
    adjoint_A, adjoint_RHS = dl.assemble_system(a_adj, L_adj)
    dl.solve(adjoint_A, p.vector(), adjoint_RHS)

    # Evaluate gradient
    MG = dl.assemble(grad_misfit + grad_reg)
    dl.solve(M, g, MG)
    grad2 = g.inner(MG)
    gradnorm = np.sqrt(grad2)

    # Set CG tolerance
    if iter == 1:
        gradnorm_ini = gradnorm
    tolcg = min(0.5, np.sqrt(gradnorm/gradnorm_ini))

    # Assemble matrices
    C = dl.assemble(C_varf)
    Wum = dl.assemble(Wum_varf)
    Wmm = dl.assemble(Wmm_varf)

    # Track if we're using Gauss-Newton
    use_gauss_newton = (iter < 10)
    if use_gauss_newton:
        gauss_newton_iters += 1

    # Define Hessian apply operator
    Hess_Apply = HessianOperator(R, Wmm, C, state_A, adjoint_A, W, Wum, use_gaussnewton=use_gauss_newton)
    P = R + 0.1*beta * M
    Psolver = dl.PETScKrylovSolver("cg", hp.amg_method())
    Psolver.set_operator(P)

    solver = hp.CGSolverSteihaug()
    solver.set_operator(Hess_Apply)
    solver.set_preconditioner(Psolver)
    solver.parameters["rel_tolerance"] = tolcg
    solver.parameters["zero_initial_guess"] = True
    solver.parameters["print_level"] = -1

    # Solve Newton system
    solver.solve(m_delta, -MG)
    total_cg_iter += Hess_Apply.cgiter

    # Line search
    alpha = 1
    descent = 0
    no_backtrack = 0
    m_prev.assign(m)
    while descent == 0 and no_backtrack < 10:
        m.vector().axpy(alpha, m_delta)
        state_A, state_b = dl.assemble_system(a_state, L_state)
        dl.solve(state_A, u.vector(), state_b)
        [cost_new, misfit_new, reg_new] = cost(u, d, m, beta)

        if cost_new < cost_old + alpha * c * MG.inner(m_delta):
            cost_old = cost_new
            descent = 1
        else:
            no_backtrack += 1
            alpha *= 0.5
            m.assign(m_prev)

    graddir = np.sqrt(-MG.inner(m_delta))

```

```

print(f"{iter:2d} {' '} {Hess_Apply.cgiter:2d} {' '} {cost_new:8.5e} {' '}
      {misfit_new:8.5e} {' '} {reg_new:8.5e} {' '} {graddir:8.5e} {' '} {gradnorm:8.5e} {' '} {alpha:5.2f} {' '} {tolcg:5.3e}")

if plot_on:
    nb.multil_plot([m,u,p], ["m","u","p"], same_colorbar=False)
    plt.show()

# Check convergence
if gradnorm < tol and iter > 1:
    converged = True
    print(f"Newton's method converged in {iter} iterations")
    print(f"Total number of CG iterations: {total_cg_iter}")
    print(f"Gauss-Newton iterations: {gauss_newton_iters}")

    iter += 1

if not converged:
    print(f"Newton's method did not converge in {maxiter} iterations")

# Compute final difference with true solution
m_diff = dl.Function(Vm)
m_diff.vector()[:] = mtrue.vector()[:] - m.vector()[:]

# Plot results
plt.figure(figsize=(15, 5))
nb.multil_plot(
    [mtrue, m, m_diff],
    ["True m", f"Recovered m ( $\beta={optimal\_beta:.1e}$ )", "Difference"],
    same_colorbar=False
)
plt.suptitle(f"Refinement level {nrefinements}")
plt.show()

# Plot beta vs error norm
plt.figure(figsize=(8, 5))
plt.semilogx(beta_values, [res['m_diff_norm'] for res in beta_test_results], 'b.-')
plt.semilogx(optimal_beta, beta_test_results[optimal_idx]['m_diff_norm'], 'ro', label='Optimal beta')
plt.xlabel('Regularization parameter  $\beta$ ')
plt.ylabel('Error norm ( $m_{true} - m$ )')
plt.title(f'Error vs Regularization (Refinement {nrefinements})')
plt.grid(True, which="both", ls="-")
plt.legend()
plt.show()

# Store results
results.append({
    'refinement': nrefinements,
    'optimal_beta': optimal_beta,
    'total_iterations': iter-1,
    'gauss_newton_iterations': gauss_newton_iters,
    'total_cg_iterations': total_cg_iter,
    'final_cost': cost_new,
    'final_gradnorm': gradnorm,
    'm_diff_norm': dl.norm(m_diff, 'L2'),
    'mesh_size': mesh.num_cells()
})

# Print summary of results
print("\n\n=== Summary of Results ===")
print("Refinement | Mesh Size | Optimal  $\beta$  | Total Iters | GN Iters | Total CG Iters | Final Cost | Final GradNorm | m_diff L2")
for res in results:
    print(f"{res['refinement']:10d} | {res['mesh_size']:9d} | {res['optimal_beta']:8.2e} | {res['total_iterations']:11d} | "
          f"{res['gauss_newton_iterations']:8d} | {res['total_cg_iterations']:13d} | {res['final_cost']:10.2e} | "
          f"{res['final_gradnorm']:12.2e} | {res['m_diff_norm']:8.2e}")

```